Who Bears the Latency Tax? Incidence of the HFT Arms Race on a Model-Checked Limit Order Book

Atharva Patil

Xavier Institute of Engineering

Abstract

Budish, Cramton, and Shim argue that the continuous limit order book's latency arms race taxes liquidity providers, but which participants bear it is an open empirical question, not a settled implication. We map incidence over snipe-to-quote ratio, HFT count, maker count, and demand elasticity on a C++ limit order book model-checked in TLA+, with the welfare decomposition closing to machine precision. Our headline result is within the liquidity-providing class: at the snipe-to-depth ratio measured on 86 hours of BTCUSDT-perpetual data, the fastest maker gains in every cell tested at two and three makers, while the slowest funds it and the snipers. The class-level answer is structure-dependent — a single maker inverts the prediction and profits, through reduced pickoff losses rather than higher noise revenue, a gain that survives unit-elastic demand, while competitive supply restores it. Frequent batch auctions recover maker welfare above a minimum clearing cadence.

1. Introduction

1.1 The arms-race argument

Budish, Cramton, and Shim (2015, hereafter BCS) argue that the continuous limit order book—the dominant design of modern equity and futures markets—contains a structural flaw that has nothing to do with any individual participant's conduct. Because the book processes messages serially in continuous time, each arrival of public information creates a fleeting, mechanical arbitrage opportunity: whichever trader reacts first can pick off the stale quotes of liquidity providers before those quotes can be revised. The competition to be first is a contest over speed rather than price, and in equilibrium it dissipates real resources into a socially wasteful arms race while transferring rent from liquidity providers (and, indirectly, the investors they serve) to the fastest "snipers." Crucially, the resulting latency tax widens effective spreads and degrades liquidity without improving price discovery; it is a prisoner's dilemma baked into the market's microstructure. BCS propose frequent batch auctions—discrete-time, uniform-price clearing—as a design remedy that converts competition on speed back into competition on price, neutralizing the mechanical sniping advantage.

1.2 Why a model-checked substrate matters

Agent-based market models carry a standing internal-validity problem: when an emergent pathology appears, it may reflect the economics under study or a defect in the bespoke, unverified matching machinery the agents trade on. We remove that confound by running the entire BCS mechanism — latency-differentiated observation of a common fundamental and a submission-timing race — as a Python scheduler atop a C++ limit order book whose matching core is model-checked in TLA+, reached through a pybind11 bridge, with a welfare identity that closes to machine precision on every run. §3.1 states the verification's exact scope and its two boundaries; §3.2 reports the differential-testing episode that shows why the scope, and not the phrase "model-checked," is what carries the weight.

1.3 Preview of findings

This paper's claim concerns the *incidence* of the latency tax — which class of participant actually bears it — and §§4.7–4.8 pose and answer that question jointly. The four experiments preceding them are not four co-equal findings. They establish the properties an incidence claim has to be able to assume, each run first at a pre-registered operating point chosen for mechanism clarity and then re-run at parameters fitted to real order flow.

- **The transfer is real and the accounting closes (Experiment 1, §4.1).** With latency-advantaged HFTs present we measure a positive HFT rent of \$34.70 (95% CI [26.25, 44.80]) that is a near-exact zero-sum transfer from the market maker and noise traders, the welfare residual closing to machine zero (mean 2.8×10^{-12}), while liquidity gaps rise from 2.84 [2.47, 3.26] in the matched no-HFT baseline to 9.70 [9.15, 10.27] in treatment; the result holds in all nine cells of a 3×3 sensitivity-by-decay grid. Absent a decomposition that closes, any statement about who bears the tax would be an accounting artifact.
- **The mechanism carries no hidden threshold (Experiment 2, §4.2).** Rent rises with market-maker staleness at approximately \$10.46 per tick, and the data do not distinguish a linear specification from a segmented one ($\Delta\text{BIC} = -0.17$, inside the range of indifference), with rent first becoming statistically distinguishable from zero at roughly two ticks. A boundary mapped across a smooth mechanism is a boundary rather than an artifact of a discontinuity.
- **Aggregate rent is fixed, and competition surfaces as fragility (Experiment 3, §4.3).** As competitors enter, per-HFT rent collapses while *total* extracted rent stays flat (mean \$34.4, range \$33.70–\$34.93 across $k = 1 \dots 21$); the marginal social cost of added speed competition appears not as rising aggregate rent but as rising fragility, liquidity gaps growing approximately linearly in k (fitted $\approx 2.1k + 3.7$, $R^2 = 0.997$), roughly 17× by 21 HFTs. This is what makes HFT count a real second axis of the incidence surface rather than a proxy for aggregate pressure.
- **The clearing rule drives the outcome (Experiment 4, §4.4).** Switching the engine to frequent batch-auction clearing recovers market-maker welfare monotonically over intervals ≥ 5 — about 97% of the loss by an interval of 500 (MM loss \$28.5 → \$0.7) — and this MM-recovery signal is the robust one; HFT-rent point estimates also fall, but only the interval-5 arm attains a rent interval clearing zero. An interval of 1 backfires (MM loss worsens to \$41.3, about 137 gaps), so the remedy helps only away from that degenerate corner.
- **The environment is disciplined by real order flow (§3.7, §4.6).** Fitting to 27 days of BTCUSDT-perpetual order flow and re-running all four experiments at the fitted parameters leaves every qualitative finding intact, and supplies the magnitude the operating point cannot: normalised by traded notional, HFT rent is 0.125 bp at the operating point and 4.42 bp calibrated. The calibrated figure is an upper bound, because a single latency-disadvantaged maker absorbs the whole race.

With those four properties in hand — a decomposition that closes, a mechanism without thresholds, a fixed rent pool whose social cost surfaces as fragility, and a clearing rule demonstrably driving the outcome — and with the environment disciplined by real flow, the incidence question becomes answerable rather than rhetorical. Two sections answer it, and they must be read together.

- **Incidence inverts under single-maker supply (§4.7).** Mapping a 63-cell surface over snipe-to-quote size ratio and HFT count, holding liquidity supply at a single maker throughout, yields a monotone regime boundary: the maker bears the tax when snipes are large relative to quoted depth, and at small snipe sizes it gains instead, with noise traders

bearing the entire transfer. Decomposing that gain shows it is loss reduction rather than revenue capture: the maker's noise-flow revenue *falls* when it widens, since widening sheds volume faster than it recovers edge per fill, and the gain comes from the accompanying drop in adverse-fill losses, with inventory carry supplying a third to a half of the net effect (§4.7). The ratio measured on 86 complete book-and-trade hours of the perpetual, 0.0072 at the median, falls inside that maker-gains region at every HFT count we test.

- **Competitive supply restores BCS, and within-class incidence is redistributive (§4.8).** Re-running the surface at maker counts $M \in \{1, 2, 3, 5\}$ — 441 cells, the $M = 1$ column nesting §4.7 to zero absolute difference — the maker-gains region at the tape band vanishes at $M = 2$, the smallest competitive case, and the maker's PnL delta turns negative with its interval excluding zero. The noise-trader toll reverses in step, so competition reinstates the incidence BCS predicts rather than merely cancelling the inversion. But that aggregate is a sum over unlike makers, and decomposing it at $n = 100$ shows every arm is redistributive: the fastest maker still gains in all eighteen tape-band cells while the slowest funds both it and the snipers. A companion robustness experiment (§4.8) relaxes the inelastic noise-trader assumption up to unit elasticity and leaves that picture intact — seventeen of eighteen tape-band cells still classify as maker gains, the one loss turning indeterminate rather than reversing.

The result is therefore a boundary over two structural axes rather than a verdict on a class. Whether liquidity providers bear the latency tax depends on how many of them supply the book and on how the class is weighted; at the measured ratio, slow makers bear it and fast makers do not. §§4.1–4.6 are what license that reading.

1.4 Contribution statement

This paper makes five contributions, and we lead with the one we regard as most consequential.

First, and most consequentially, it identifies the *incidence* of the latency tax as an empirical question distinct from its existence, and maps it over two structural axes rather than answering it with a verdict. Sweeping snipe-to-quote size ratio against HFT count, holding liquidity supply at a single maker, yields a monotone regime boundary: the maker bears the tax when snipes are large relative to quoted depth and gains at small ones — not because widening earns it more from noise flow, which the leg decomposition shows falls, but because widening reduces its adverse-fill losses, with inventory carry supplying a third to a half of the net. The ratio measured on 86 complete book-and-trade hours of BTCUSDT-perpetual data, 0.0072 at the median, falls inside that maker-gains region at every HFT count we test, so under single-maker supply the BCS prediction inverts (§4.7). Adding the maker-count axis then reverses it back: at two competing makers the maker-gains region at the tape band is gone, the maker's delta turns negative with its interval excluding zero, and the noise-trader toll falls in step, so competitive supply reinstates the incidence BCS predicts (§4.8). The two sections are one result. Incidence is a function of market structure — of snipe size, sniper count, and the number of makers supplying the book — and neither BCS nor the direct message-level measurement of Aquilina, Budish, and O'Neill (2022) treats it as one.

Second, and cutting across the first, incidence within the liquidity-providing class is redistributive, so the class-level question is not well posed without a weighting. Decomposing the aggregate maker PnL at $n = 100$ across the tape-measured band shows the summed figure conceals opposite signs in every arm tested: at two makers the leading one gains in all eighteen band cells with its interval excluding zero, while the slowest pays in all eighteen and funds both it and the snipers (§4.8). The aggregate turns negative because the slow maker's loss exceeds the fast maker's gain, not because liquidity provision loses as an activity. The defensible statement is accordingly narrower than either the inversion or its reversal: *slow* liquidity providers

bear the latency tax. That statement was then tested against the assumption most likely to overturn it and survived: making uninformed demand spread-elastic up to unit elasticity costs the maker-gains region one of eighteen tape-band cells, and the cell lost turns indeterminate rather than reversing (§4.8). We also report an unexplained anomaly against this picture — at three calibrated makers the ordering is not monotone in latency, the middle maker being the sole consistent winner — rather than fitting a mechanism to it.

Third, and as supporting methodology rather than a headline claim, the substrate. The model-checked specification is not itself the contribution — TLC certifies a design at bounded scope, not the C++ that runs (§3.1). What does the work is *differential testing against the verified engine*, and §3.2 shows why that distinction is operative rather than pedantic: a Python re-implementation of the clearing rule, checked against the C++ source line by line and described in an earlier draft as faithful, cleared a volume-tied auction at a different price the first time the two were run against the same book, and re-routing Experiment 4 through the engine moved three of seven batch cells across zero in both significance and sign. Paired with a welfare identity that closes to machine precision in every cell we report — sixty-three single-maker, 441 maker-count, 189 elasticity — that removes design-level matching error and accounting leakage as explanations of either the inversion or its reversal, which is the specific way an agent-based incidence claim would otherwise fail.

Fourth, the welfare decomposition of Experiments 1–3 establishes the properties that make the §§4.7–4.8 surface a credible extension rather than a standalone claim: the arms race is a transfer rather than value destruction, its cost scales smoothly in latency with no phase transition, and as competitors multiply the aggregate rent pool stays fixed while the social cost surfaces as rising fragility. A boundary mapped across a mechanism with those properties is a boundary, not an artifact of a discontinuity or of a rent pool that grows with the axis being swept.

Fifth, the batch-auction treatment supplies the policy-relevant result the validated harness makes available: the BCS remedy restores liquidity-provider welfare monotonically above a minimum clearing cadence and backfires below it, the interval-1 arm worsening the maker's loss rather than repairing it. MM recovery is the robust signal there; the rent-reduction point estimates are not.

The negative and nuanced results — linear rather than threshold scaling, fragility rather than rising rent, MM recovery as the clean remedy signal, and the failure of incidence to reduce to aggregate snipe pressure — are integral to the contribution.

2. Related Literature

This paper sits at the intersection of five strands of work: the market-design theory of the HFT arms race and its direct measurement, the theory of speed and exchange design, the game theory of predatory liquidity provision, the empirical and physics-of-markets literature on endogenous fragility, and the methodological tradition of agent-based modeling. We synthesize each in turn and then state our differentiator.

The arms race and its remedy. Our central reference is Budish, Cramton, and Shim (2015), who reframe latency competition not as a behaviour to be regulated but as a symptom of the continuous limit order book's serial-processing design, and who propose frequent batch auctions as the corrective. Our work is a controlled experimental instantiation of their model: we build the latency-differentiated observation structure and submission-timing race they describe, reproduce the predicted treatment effects, and test their remedy by switching the clearing rule of the same verified engine, supplying a closed welfare decomposition their reduced-form empirics cannot directly observe. The race has since been measured directly: Aquilina, Budish, and O'Neill (2022), using message-level data that records failed race attempts alongside successful ones,

find races roughly once per minute per FTSE 100 symbol, decided by microseconds, summing to a latency-arbitrage tax of about 0.4 basis points of volume. Their measurement and our experiment are two views of one transfer — they infer it from exchange messages, we observe every leg of it under a conserved accounting identity. Menkveld (2016) surveys the ground between, cataloguing HFT's double role as liquidity supplier and source of adverse selection, and noting how little of the literature settles the net welfare sign for any participant class. The incidence question of §4.7 sharpens exactly that unsettled sign.

Speed and exchange design. Menkveld and Zoican (2017) show that exchange speed is not neutral infrastructure: a faster engine accelerates snipers and makers alike, raising the frequency of speed duels, so lower latency can *worsen* liquidity when news arrival is high relative to liquidity-motivated trade. Our degenerate interval-1 cell, where the most aggressive clearing cadence amplifies the pathology it was meant to cure, is a discrete-clearing cousin of that comparative static. Baldauf and Mollner (2020) compare design responses — batch auctions and asymmetric speed bumps — and show both blunt the sniping externality with different effects on price discovery; we test the first experimentally, and the second is implementable in the latency scheduler without touching the matcher. Du and Zhu (2017) ask not whether discrete clearing helps but what frequency is optimal, batching longer letting more information accumulate into each price while delaying incorporation and thinning participation; Experiment 4 is the experimental counterpart of that trade-off, and what we add is a closed welfare decomposition and a fragility channel an optimal-frequency calculation does not price. Hoffmann (2014) supplies the equilibrium logic our maker executes mechanically: fast traders' ability to revise ahead of news concentrates picking-off risk on slow limit-order submitters, who rationally quote less aggressively. Our maker's adverse-selection widening (§3.3) is that response imposed as a rule rather than derived, which is why its parameterisation is a limitation we test directly (§5.2).

Predatory versus cooperative liquidity. Carlin, Lobo, and Viswanathan (2007) provide the strategic micro-foundation for endogenous liquidity evaporation: when cooperation among strategic traders breaks down, agents race to trade ahead of a distressed counterparty and amplify price impact, producing crises that owe nothing to exogenous shocks. The BCS sniping mechanism is a latency-mediated instance of that predatory equilibrium; we differ in running an explicit agent-based experiment with a closed accounting identity rather than a stylized analytic game. Foucault, Kozhan, and Tham (2017) isolate the informational subset empirically, showing that *toxic* arbitrages — those triggered by news rather than transient imbalance — impose the adverse selection that widens spreads. Their toxic/non-toxic split is the empirical analogue of our maker's adverse-fill classifier (§3.3), and the incidence result turns on how much the widening it triggers cuts the toxic side's take relative to what it costs on the non-toxic side.

Adverse selection and the maker's spread. The tradeoff our mechanism runs through predates the latency literature. Copeland and Galai (1983) model the quoted spread as the outcome of a maker's choice between revenue collected from uninformed traders and losses to better-informed ones, so that widening is simultaneously a defence against the second and a tax on the first — which is precisely the two-legged structure the decomposition table of §4.7 measures, and their framing is why we report the legs separately rather than only their difference. Glosten and Milgrom (1985) make the adverse-selection component of the spread explicit as an information cost: even a competitive, risk-neutral, zero-inventory-cost maker must quote a strictly positive spread, because each trade carries the possibility that the counterparty knows more, and the spread is the price of that possibility. Our maker is a mechanical realization of that logic in the time domain rather than the information domain — its disadvantage is latency rather than private information — but the object it is defending against is the same, and the widening rule of §3.3 is a discrete-time analogue of their information component adjusting to the observed rate of informed trade.

Endogenous fragility and flash crashes. Kirilenko, Kyle, Samadi, and Tuzun (2017) dissect the May 2010 Flash Crash from audit-trail data and find that HFTs, while not the trigger, consumed liquidity and passed inventory among themselves at the critical moment, accelerating the decline. Experiment 3 speaks directly to that episode by asking whether such fragility can arise from rational HFT behaviour alone, with no exogenous shock; we find it grows with the number of competing HFTs, and where Kirilenko et al. perform forensics on one historical event we offer a reproducible counterfactual in which HFT count, latency, and market design are knobs. Filimonov and Sornette (2012) supply the diagnostic we borrow: the branching ratio of a self-exciting Hawkes process as a measure of reflexivity, which spikes toward criticality around fragile periods. We estimate it on simulated order flow via a Hawkes bridge, the advantage being that the data-generating process is known, so the ratio can be tied causally to HFT count and latency rather than inferred post hoc (§4.5).

Agent-based modeling and its verifiability gap. Methodologically, our work extends the agent-based computational finance tradition (Farmer and Foley, 2009; LeBaron, 2006), which argues that bottom-up simulation of heterogeneous, boundedly rational agents can generate emergent, out-of-equilibrium phenomena—fat tails, volatility clustering, bubbles and crashes—that representative-agent equilibrium models cannot. That tradition's well-known weaknesses are sensitivity to ad hoc behavioral assumptions, weak calibration discipline, and limited reproducibility or verifiability of the simulation machinery itself. The closest agent-based precursor to this paper is Wah and Wellman (2013), who build a tactical agent-based model of latency arbitrage across fragmented markets and find that a latency arbitrageur profits at the expense of other traders while degrading allocative efficiency — qualitatively the transfer we decompose, established a decade before us. Our departure from that work is therefore not the agent-based method, which it had already applied to this mechanism, but the substrate the method runs on and the question we ask of its output. We inherit the tradition's strength (emergent crash dynamics from rational behavior) while directly confronting its central weakness: rather than building on a bespoke, unverified simulator whose matching logic could itself generate the observed artifacts, we drive a TLA+-verified C++ matching engine, and we impose calibration and welfare-accounting discipline in the form of a closed, near-zero welfare residual.

Differentiator. Agent-based study of latency arbitrage is not itself new: Wah and Wellman (2013) modelled it and recovered the same qualitative transfer, so novelty of approach is not what we claim. What distinguishes this paper is, to our knowledge, the first experimental test of the BCS welfare decomposition on a *model-checked* matching engine, and the first treatment of the latency tax's *incidence* as a question distinct from its existence. BCS is theory plus reduced-form empirics, since joined by the direct message-level measurement of Aquilina, Budish, and O'Neill (2022); the agent-based tradition, Wah and Wellman included, runs on bespoke simulators whose matching layer is a candidate source of the very effects under study. Here the entire BCS mechanism lives in a Python scheduler atop a deterministic matcher whose continuous-matching specification was model-checked with zero invariant violations (per the engine's TLC verification log, `docs/Verification.md`), reached through a pybind11 bridge, so emergent arms-race dynamics, liquidity gaps, and crash behavior cannot plausibly be artifacts of the matching design (in the bounded sense of §1.2). On that substrate we decompose welfare into market-maker, noise-trader, and HFT components, show the arms race is a transfer that closes to a machine-precision zero-sum residual, trace the mechanism across latency and HFT-count sweeps (with the findings that scaling is linear rather than threshold and that aggregate rent is flat while fragility rises), and confirm the BCS remedy experimentally via the batch-auction treatment, with a placebo-validated endogeneity diagnostic in the Filimonov–Sornette spirit that returns a null at our operating point. It is the joint presence of a verified engine, a closed welfare identity, and an experimental confirmation of both the pathology and its proposed cure that situates this paper relative to the literature it builds on.

3. Model and Experimental Design

This section describes the simulation apparatus. The design rests on a deliberate separation of concerns: a single model-checked C++ matching engine adjudicates every trade — continuous matching and batch-auction uncrossing alike — while a Python layer above it supplies the agents, the latency model, and the batch-auction cadence. The verified engine is the part we trust to be correct; the Python layer is the part we vary. The boundary between them is the pybind11 bridge, and it is drawn precisely where the engine's formal guarantees end.

3.1 The Matching Engine (C++20, TLA+-verified)

All trades in the continuous arm are produced by one C++ matching engine, `OrderMatcher::MatchingEngine`, the same object that is the subject of the project's TLA+ specification suite. In the BCS harness the engine is run in **synchronous** mode: `EngineHarness::start()` calls `engine_.start()` rather than `startAsync()`, so `submitOrder()` dispatches matching on the calling thread and the `OrderBook`'s `EventListener` fires inline. Draining trades immediately after a submit therefore returns exactly the fills that order produced, which keeps the harness deterministic and free of cross-thread or GIL concerns (`bcs_research/bindings/engine_bindings.cpp`, lines 5–16). The engine offers price-time priority matching, best bid/ask, depth snapshots, VWAP, and an optional circuit breaker; the breaker is disabled in this harness via `setCircuitBreakerThreshold(1e18)` when `start(disable_circuit_breaker=True)` so that simulation control, not an exchange-level halt, governs the dynamics (lines 81–83). Prices are fixed-point: a quoted price of 100.00 is `100 * PRICE_PRECISION`, with `PRICE_PRECISION` re-exported as `bcs_engine.PRICE_PRECISION` (line 186).

Critically, **the engine has no notion of agent latency.** The comment at `engine_bindings.cpp` lines 15–16 states this explicitly: the entire BCS latency/race mechanism lives in the Python scheduler above this layer. The engine's job is only to be a correct continuous double-auction matcher, and that correctness is what the spec suite certifies.

The directly relevant specification is **`Auction.tla`** (`spec/Auction.tla`), which models the engine's session lifecycle over the seven legal `TradingState`s (`PreOpen`, `AuctionOpen`, `Continuous`, `AuctionClose`, `PostClose`, `Halted`, `VolatilityAuction`) and the discrete uncross that is the only way trades emerge from an auction state. Under `Auction.cfg` (`BROKEN_NO_REOPEN=FALSE`, `MaxOrders=3`, `MaxEvents=6`, `Prices={100,200}`), TLC checks the following **invariants**:

- **`TypeOK`** — the type invariant.
- **`ValidState`** — every reachable state is one of the seven legal `TradingState`s.
- **`NoMatchOutsideContinuousExceptUncross`** — no trade prints in an auction, halt, or closed state except a discrete uncross whose source is a true auction state (`PreOpen`, `AuctionClose`, or `VolatilityAuction`). Each trade carries an `inState` history field, which gives the invariant teeth.
- **`SingleClearingPrice`** — every uncross firing produces trades at exactly one price drawn from `Prices`. This is the code path the batch arm now clears through via the bridge's `uncross` (§3.2).
- **`NoTradeWhileHalted`** — no trade record may carry `inState=Halted`.

As temporal/action **properties** it checks **`ValidTransitions`** (`[][ state'=state \/ <<state,state'>> \in LegalEdges ]_state` — every step is either a self-loop or a legal directed edge of the session state machine) and the **liveness** property **`VolatilityEventuallyReopens`** (`[](state=VolatilityAuction => <>(state \in {Continuous,Halted}))`, proved under weak fairness `WF_vars(ReopeningUncross)`). Non-vacuity is established by a companion `AuctionBroken.cfg` (`BROKEN_NO_REOPEN=TRUE`) that disables the reopening uncross so TLC reports a liveness counterexample, confirming the property is not vacuously true.

We did not recover a model-checked state count for `Auction.tla`: a `states/` directory and TLC trace (`.bin`) artifacts exist in the spec tree, but we did not parse them, and no state count is reported for it. The large reachable-state count cited elsewhere for the engine (§1.2) pertains to `MatchingEngine.tla` and is recorded in `docs/Verification.md`, not derived from `Auction.tla`.

`Auction.tla` sits within a broader suite (`spec/`) that verifies the surrounding engine and infrastructure. We summarize the checked properties rather than restate every configuration:

Seven further specifications cover the matching core, replication, queue, and session lifecycle; their checked properties and configuration parameters are documented in the paper's replication archive. The one number we cite in the main body is for `MatchingEngine.tla`, whose deepest exhaustive configuration (`MaxOrders=4`, `MaxTime=2`, two prices) explores 171,187,419 distinct reachable states to completion with zero invariant violations, checking `FIFOExecution` and `MatchingConservation` for the continuous cross action alongside the book-management invariants. A companion broken configuration admits non-FIFO fills so TLC returns a counterexample, establishing non-vacuity.

The payoff is that any artifact the simulation reports — a fill, a price, a snapshot, a zero-sum residual — is produced by a matcher whose price-time priority, FIFO preservation, non-negativity, and single-clearing-price behaviour have been model-checked, not merely tested.

Two boundaries on that verification deserve equal precision, because "model-checked" is easy to over-read. TLC's guarantee is exhaustive only within the stated bounds — a handful of orders, two prices, short horizons — so it certifies the matching *design* against its invariants at bounded scope rather than proving unbounded correctness. And the object verified is the TLA+ specification, not the C++ source: no refinement proof links the two, and the correspondence rests on construction and conformance testing. Wherever this paper says "verified engine," both qualifications are attached. What the provenance buys, and what the internal-validity argument uses, is the elimination of design-level matching errors — the kind that could manufacture spurious gaps or transfers — within the checked scope, on the code path every trade in both arms traverses.

3.2 The pybind11 Bridge

A pybind11 module named `bcs_engine` bridges the verified C++ engine to Python (`engine_bindings.cpp`, `PYBIND11_MODULE` at line 212). It exposes a single entry class, `EngineHarness`, with: `add_symbol` (which must precede `start()`; `start()` freezes the book registry, after which `add_symbol` throws/no-ops, lines 64–70), `start(disable_circuit_breaker=True)`, `stop`, `submit_order(symbol, order_id, participant_id, side, price, quantity, order_type=Limit, tif=GTC)`, `submit_cancel`, `best_bid`, `best_ask` (with `INT64_MAX` normalized to 0 when the ask side is empty, lines 129–134), `mid` (0 if either side is empty), `drain_trades`, `snapshot(depth=10)`, `vwap`, `trade_count`, and — added to close the matcher seam described below — `uncross` (one uniform-price call-auction clearing of the resting book) together with `set_trading_state/trading_state` over the session states of `Auction.tla` (lines 163–189). Fills are captured by a per-symbol `TradeCollector` that buffers every trade on the book and clears on `drain_trades`; Python attributes each fill to agents by `buyer_id/seller_id`. The enums `Side`, `OrderType` (`Limit/Market/IOC/FOK/PostOnly`), `TimeInForce` (`GTC/GTD/DAY`), and `RejectReason`, plus read-only `Trade`, `PriceLevel`, and `MarketDataSnapshot` classes, are exposed alongside.

Python agents drive the verified engine entirely through this surface: each tick they emit orders and cancels, the scheduler submits them via `submit_order/submit_cancel`, and because the engine is synchronous, the scheduler drains the resulting fills inline and routes them back to the

agents. The agents never touch C++ state directly; the bridge is the only channel, which is what lets the verified matcher remain the sole adjudicator of trades.

An earlier revision of this harness carried a methodological seam here: the bridge did not expose `uncross`, so the frequent batch (call) auction could not be run through the engine at all and was instead re-implemented in Python (`discover_uncross_price`), replicating `OrderBook::discoverUncrossPrice` by source inspection. That seam is now closed. The bridge exposes `uncross`, `set_trading_state`, and `trading_state`; the batch arm (`BatchAuctionScheduler`) parks the book in `AuctionOpen` — where orders accumulate instead of matching on arrival — collects the window's orders, and clears through the engine's own `uncross`, the code path `Auction.tla` proves `SingleClearingPrice` over. Both arms of Experiment 4 therefore run on the same verified matcher, and the batch arm's fills are engine fills, drained through the same `TradeCollector` as continuous fills.

Closing the seam produced a finding in its own right. The retired Python re-implementation had been checked against the C++ source line by line, and an earlier draft of this paper described it as replicating the rule "faithfully" — a claim we noted was source-verified rather than results-verified. When the two implementations could finally be run against the same book, they disagreed. On a book where two prices tie on maximum executable volume (bids 102×50 and 101×30 against asks 99×40 and 100×60, so 80 units clear at either 100 or 101), the Python rule clears at 100 and the verified engine clears at 101 — a divergence inside the tie-break cascade, invisible to any volume-based check and material to every metric that depends on execution price, rent above all. The divergence is pinned in the test suite (`tests/test_batch_auction.py::test_python_matcher_divergence_pinned`) so it cannot regress silently, and `discover_uncross_price` is retained only as a cross-check. The episode is a compact argument for this paper's central methodological claim: a re-implementation that is source-verified can still differ from the verified artifact the first time it is results-verified.

The divergence is causally localized to the `uncross` path, and the experiment contains its own control. A natural objection to the preceding paragraph is that two runs of a stochastic simulation differ for many reasons, so a changed result need not indict the matcher at all. The design answers this objection without requiring us to argue it. Experiment 4's long grid comprises seven batch arms and one continuous arm, and the continuous arm does not call `uncross` — it matches on arrival, exactly as Experiments 1–3 do. When the grid is re-run with the batch clearing rerouted from the retired Python re-implementation to the engine, the continuous arm returns 578.1 [449, 709], **byte-identical** to its pre-port value, while three batch cells move across zero in both significance and sign (§5.2). Seeds, fundamental paths, agent realizations, run length and every environment parameter are held fixed across the two runs; the single thing that differs is which implementation clears the batch. A control arm that is bit-for-bit unchanged therefore rules out seed variation, drift in the agent code, and environment differences as explanations, and confines the effect to the code path that was substituted. This is a stronger form of evidence than the tie-break counterexample above: the counterexample shows the two rules *can* disagree on a constructed book, whereas the byte-identical control shows the disagreement *did* propagate into published economic quantities and nothing else plausibly did. We draw the general methodological lesson in §5.2, but state the identification here, in the methods, because it is a property of the experimental design rather than an interpretation of the results.

3.3 Agents

All agents are Python objects driven by the scheduler, sharing one interface — `act(t, market, fundamental)` returns a list of `Action`s and `on_fill(trade, is_buyer)` receives routed fills — with each holding its own cash and inventory for the welfare accounting of §3.5. Constants are

in the replication archive; what matters here is each agent's role and decision rule.

`FundamentalValueProcess` is the exogenous true value $V(t)$, a discrete random walk in fixed-point price space. Its latency-delayed read returns the most recent V at or before $t-\ell$, and that delay is the BCS information gap separating a fast agent from a slow one; the undelayed value is used only for the maker's adverse-fill test.

`BCSMarketMaker` is an adverse-selection-aware two-sided quoter. On each requote tick it decays its half-spread toward base, observes V with its larger latency — so its quotes are stale — and posts a bid and ask around that stale view, skewed by inventory. Two coupled defences drive the mechanism. It widens *only* on adverse fills, meaning fills where it bought above or sold below the true current V; noise fills sitting inside the quoted spread leave it untouched. And liquidity thins as it widens: resting size scales as `base_half_spread / current_half_spread`, so spread and depth are one coupled state rather than two independent knobs.

`HFTAgent` is a latency-advantaged stale-quote sniper. It observes V with little or no delay, and fires an aggressive IOC when the gap between V and the resting quote exceeds its transaction cost. The race is a random submission jitter added to each order's submit time: with several HFTs the lowest draw wins and the others find the quote already gone. Its counterparty is the slower maker, and that latency gap is the BCS arbitrage.

`NoiseTrader` is the uninformed liquidity taker and the control population. Each tick it crosses the spread with fixed probability on a randomly chosen side, carrying no fundamental signal and never sniping, so it pays the half-spread and its expected PnL is negative by construction. Arrival is inelastic in spread width in every experiment through the competition arm; the elastic-demand arm relaxes exactly this, scaling the crossing probability by $(s_{\text{base}}/s)^{\alpha}$, with $\alpha = 0$ recovering the inelastic path bit-for-bit.

3.4 The BCS Mechanism

The `LatencyScheduler` (`bcs_research/simulation/scheduler.py`) is the race substrate, supplying the latency model the C++ engine deliberately lacks. Each tick, advancing by `dt_us`, it steps the fundamental, resets per-tick volume and signed volume, lets every agent emit `Actions`, and enqueues them in a heap keyed `(submit_at, seq)`. It then pops and submits every action whose `submit_at` has arrived, in submit-time order, draining trades after each submission and routing fills back to the buyer and seller by `participant_id`. The BCS submission race is exactly many agents stamping near-identical `submit_at` values, resolved here in arrival-time order through the heap: there is no special-case race logic, only priority over arrival time, which is what makes the race an emergent consequence of the latency model rather than a scripted outcome. The scheduler records a per-tick snapshot — best bid, best ask, mid, fundamental, tick volume, signed volume — and every fill into a `SimResult`.

The endogenous-instability signature of this mechanism is not a mid dislocation but a **liquidity gap**: a transient one-sided or empty book, where a thinned level is emptied before the maker requotes. An episode qualifies only if it is a maximal run of non-two-sided snapshots that recovers to a two-sided book within `max_gap_us` (50,000 µs; longer runs are structural outages and are discarded) *and* is endogenous, meaning the fundamental is flat across it: $|V_{\text{hi}} - V_{\text{lo}}| < k_{\text{fund}}\,\sigma\sqrt{g+2}$ with $k_{\text{fund}} = 3$. Value-driven gaps are excluded by construction. We use this event definition because an earlier all-fills maker, which widened on every fill rather than only adverse ones, produced no HFT-specific divergence in mid prices: the instability this mechanism generates surfaces as gaps in depth, not crashes in price. ### 3.5 Metrics

All metrics are pure functions over a `SimResult` and apply unchanged to both the continuous and batch arms, since `BatchAuctionScheduler` emits the same `SimResult` shape.

Welfare decomposition and the zero-sum identity (bcs_research/metrics/welfare_decomposition.py) `decompose_welfare(result, mm_id, nt_ids, hft_ids, mark)` computes per-agent PnL as `cash + inventory * mark / PRICE_PRECISION`, marked at `mark_price` (the last two-sided mid, else the last `V`). It reports `mm_pnl`, `noise_trader_pnl` (summed over `nt_ids`), `hft_pnl`, `hft_rent` (equal to `hft_pnl` — the rent the arms race extracts), per-agent and per-second variants, and `zero_sum_residual = mm_pnl + nt_pnl + hft_pnl`. In a closed market the residual is approximately zero — the conservation check that **closes through the verified engine**: because every fill is produced by the model-checked matcher with FIFO preservation and non-negative quantities, cash and inventory are conserved exactly, and any non-trivial residual would indict the harness rather than the market. `welfare_delta` computes the treatment-minus-baseline difference for the key terms; this welfare transfer is the paper's primary treatment effect, because it moves even when quoted spreads do not.

Lagged Kyle's lambda (bcs_research/metrics/baseline_metrics.py, `compute_metrics`). `kyle_lambda` is the OLS slope (`np.polyfit` degree 1) of the *forward-window* mid change `mid[t+k] - mid[t]` (default `k = kyle_horizon_ticks = 5`) regressed on `signed_volume` at `t`, over ticks with `mid > 0`; it requires at least 3 points and `std(x) > 0`, and `kyle_lambda_r2` is the squared correlation. The forward lag, rather than the contemporaneous tick, is deliberate and documented (lines 64–70): a latency-disadvantaged maker quotes stale, so a same-tick regression would score its delayed catch-up requote as *negative* impact; measuring over the next `k` ticks captures the catch-up as the *positive* permanent impact it actually is, removing the stale-quote sign artifact.

Liquidity-gap detector (bcs_research/metrics/liquidity_gap_detector.py). As defined in §3.4, `detect_liquidity_gaps` returns the bounded, recovering, endogenous one-sided episodes; `count_liquidity_gaps` returns the count. This is the secondary endogenous-instability metric, matched to the thinning mechanism (gaps, not mid crashes), and each event records `start_t`, `recovery_t`, `duration_us`, `gap_ticks`, `side_missing`, and `v_move`.

Percentile bootstrap CI (bcs_research/metrics/bootstrap.py). `bootstrap_ci(values, n_bootstrap=2000, ci=0.95, seed=0)` is a nonparametric percentile bootstrap for the mean: it draws `(n_bootstrap, n)` resamples with replacement via `np.random.default_rng(seed)`, takes per-row means, and returns the (seed-independent) point estimate `mean`, the `lower`/`upper` bounds at the `(alpha/2, 1 - alpha/2)` percentiles, `std` (ddof=0), and `n`. A single value yields a zero-width CI. `bootstrap_ci_table` builds a `{metric_key: ci}` table from per-seed row dicts. The resampling RNG is explicitly seeded for reproducibility. The `n_bootstrap=2000` resample count and percentile method are defaults of the bootstrap module (source-level), not fields stored in the results JSON, which records only the bounds and `n`. We note that `exp1_primary.py` itself does not call `bootstrap_ci` — it aggregates per-seed rows with `statistics.fmean` — and the bootstrap module is shared infrastructure used from Experiment 2 onward; the n=100 CIs the paper reports for Experiment 1 come from the hardened run, which does use the bootstrap.

3.6 Experimental Design Summary

The primary operating point is the `CFG` dict in `bcs_research/experiments/exp1_primary.py`. Both arms share identical market parameters and differ **only in HFT presence**: the **baseline** is the `BCSMarketMaker` (latency-disadvantaged, adverse-selection-aware) plus noise traders; the **treatment** is the baseline plus `N` HFTAgents. Both run on the continuous `LatencyScheduler` through the verified engine. The headline configuration is:

The pre-registered operating point fixes 3,000 ticks at `dt_us = 1000`, eight noise traders against three HFTs, a 100-tick base spread, per-step $\sigma = 20$, a 3,000 µs maker latency against zero-latency snipers, and 20 seeds per arm with 2,000-resample bootstrap intervals. The full parameter set, including the agent-level constants of §3.3, is documented in the committed experiment

code and the paper's replication archive rather than reproduced here.

Per-seed outputs (`mm_pnl`, `nt_pnl`, `hft_rent` and `_per_sec`, `zero_sum_residual`, `liquidity_gaps`, `kyle_lambda`, `spread_ticks`, `n_trades`, `mm_max_half_spread`, `mm_min_qty`, `hft_snipes`, `hft_fills`) are written to `bcs_research/results/experiments/exp1_primary.json`; the hardened $n = 100$ results reported in §4.1 come from `exp1_hardening.py` and are written to `exp1_hardening.json`.

Around this operating point we run four one-dimensional sweeps, each isolating one axis of the mechanism:

1. **Calibration robustness (3×3).** `adverse_sensitivity` × `spread_decay` over `{0.05, 0.10, 0.20}`, confirming the result is not an artifact of the calibrated cell — HFT rent must be significantly positive and the zero-sum identity must hold in all nine cells.
2. **Latency sweep (Exp 2).** The maker's staleness gap from 0 to 20 ticks with `hft_latency = 0`, on a tick-aligned grid (latency resolves only at `dt_us` granularity), testing whether rent rises linearly or exhibits a knee/phase transition.
3. **HFT count sweep (Exp 3).** Number of HFTs over `{0, 1, 2, 3, 5, 8, 13, 21}`, separating per-HFT rent (competed away) from total rent (transfer) and from fragility (liquidity gaps).
4. **Auction-frequency sweep (Exp 4).** Batch interval over `{1, 5, 10, 25, 50, 100, 500}` plus continuous, run through `BatchAuctionScheduler` (which parks the book in an auction state and clears through the engine's own `uncross`, §3.2), measuring how much maker welfare loss the frequent batch auction recovers and at what interval.

Each sweep varies one parameter and holds the rest at the operating point, so that any change in the outcome metrics is attributable to that single axis. We note that the no-HFT baseline liquidity-gap figure is reported per experiment from its own matched run. At $n = 100$ these agree: the matched baseline anchoring the Experiment 1 headline comparison is 2.84 gaps (95% CI [2.47, 3.26]), and the Experiment 3 $k = 0$ cell that anchors the fragility ratio reports the same 2.84. An earlier draft carried a discrepancy here — an $n = 20$ baseline of 3.45 against an $n = 100$ figure of 2.84 — which we attributed to sample size; raising every experiment to $n = 100$ resolves it, confirming that reading.

3.7 Calibration to BTC order flow

To discipline the operating point against real microstructure, we calibrate the sim's *environment* parameters to Binance BTCUSDT-perpetual order flow recorded by a companion collector (20-level book snapshots at ~10/s and the aggregated-trade stream, both with nanosecond timestamps). The collector drops a few hours on most days (host sleep), which we handle by making each moment gap-safe rather than by discarding gapped days: arrival intensity accumulates each hourly file's own observed span, so missing hours leave the denominator instead of inflating it, and realized volatility keeps only returns between *adjacent* one-second buckets, so no return straddles a gap. Days enter at 18 hours of coverage in both streams; files truncated by a mid-write crash are skipped and recorded. This is not housekeeping: a whole-day span over a 21/24 day biases the arrival rate down by roughly 12%, and demanding 24/24 coverage admits only two of the days the collector has ever produced. Under the gap-safe estimators the sample is 27 days (2026-06-24 to 2026-07-21, less 2026-07-19; 37.0M trades, 4.16M snapshots, 2.27M seconds of coverage), yielding: trade arrival 16.28/s; trade sizes p50/p90/p99 of 0.003/0.177/2.01 BTC; median spread of one exchange tick (≈0.016 bp); median top-of-book depth 6.18 BTC; and 1-second realized volatility of 0.650 bp/√s (`results/calibration/targets_btcusdt.json`).

The mapping between sim ticks and wall-clock is a modelling choice we fix at 1 tick = 100 ms: the market maker's 3-tick latency then means 300 ms of quote staleness — slow but plausible

— while HFTs react within a tick, preserving the BCS fast/slow asymmetry at realistic crypto scales. Under that convention two environment parameters invert directly, and both fits verify by simulation (`calibration/fit.py`). The volatility inversion is clean: at fitted σ the baseline's simulated realized vol is 0.647 vs the target 0.650 bp/$\sqrt{\text{s}}$. The arrival inversion is not: its closed form assumes every noise order trades, but only about half the IOCs find a quote at the operating point, so the fit iterates the arrival intensity against the *simulated* trade rate to a fixed point (converging in three steps from a closed-form 0.204 to 15.94 vs 16.28/s). The fitted values are `lambda_per_tick` = 0.427 and σ = 20.6 against the pre-registered operating point's 0.15 and 20.

The volatility figure is worth dwelling on. Estimated over the full month the fitted σ lands within 3% of the value the experiments were pre-registered at, so on volatility the operating point is not data-adjacent but data-consistent. A two-day subsample passing a stricter 24/24 completeness filter gives 55.7 — nearly three times as large — because those two days were simply hot (1.76 bp/$\sqrt{\text{s}}$ against the 27-day 0.650), which is the kind of sampling artifact a small clean-data-only window invites. Arrival intensity remains genuinely under-set: the fitted per-trader rate is 2.8× the operating point's, so §§4.1–4.5 run on a market thinner in order flow than the perp, and §4.6 re-runs the full grid at the fitted parameters.

That re-run required re-calibrating one agent parameter, for a reason that generalizes to any transplant of a tuned agent into a calibrated environment. The maker's widening is applied *per adverse fill event* (`current_half_spread *= 1 + adverse_sensitivity`), so it is not scale-free in order flow: tripling the arrival rate triples the multiplicative kicks per tick while the decay pulling the spread back to base is unchanged. Measured directly, the maker's pickoff rate rises from 0.20 to 0.86 per tick between the operating point and calibrated flow, and at the pre-registered `adverse_sensitivity` = 0.1 the *no-HFT* spread settles at 5.36× base — the maker widens to its cap with no informed trader in the market at all. Any experiment run in that state would credit a control-loop artifact to the arms race. The stationary spread multiple obeys $m = (1 - P\ln(1+s)/d)^{-1}$ for pickoff rate P, sensitivity s and decay d, which predicts 5.56× at $P = 0.86$ against the 5.36× observed and 1.24× at $P = 0.20$ against 1.17×; solving it for the original calibration target locates the replacement value, and a re-run of the §3.6 sensitivity–decay grid under calibrated flow confirms it. We therefore set `adverse_sensitivity` = 0.015 for §4.6, holding `spread_decay` at its pre-registered 0.1 so that exactly one parameter moves, selected on the same criterion as the original (no-HFT spread ≈1.1× base with positive with-HFT divergence: 1.12× and +0.08, against 1.14× and +0.03 for the published cell). The 6.7× reduction in sensitivity tracks the 4.3× rise in pickoff rate, as per-event widening implies.

Two further parameters are not pinned by the data. `hft_qty` is set to the calibrated `quote_qty` (2061 units), preserving the operating point's ratio of 1.0 so that a single snipe still clears the top quote — the mechanism the race runs on; holding it at 50 instead makes a snipe take 2.4% of the book, which changes the experiment rather than rescaling it, and we report that arm separately as a robustness check. `min_quote_qty` is left at 1 and is inert at calibrated depth: the thinning floor is set by `spread_cap_mult`, with the maker bottoming out well above the nominal floor in every calibrated cell.

Equally important is what this calibration does *not* claim. Spread and depth levels are not fitted: the stylized MM quotes a ~1.5 bp median spread where the perp trades at ~0.016 bp, and its depth-to-median-trade ratio is 5× against the perp's ~2060×. The size distribution is not fitted in the experiments of §4: sim order sizes there are constants while real sizes are heavy-tailed (p99/p50 ≈ 670×). A lognormal size model for the calibrated re-run is implemented and parameterized ($\sigma_{\ln}$ = 2.80 fitted on the p50/p99 ratio; median 1 unit with a calibrated book depth of ~2061 units, matching the real depth-to-median-trade ratio), with the disclosed compromise that a two-quantile lognormal under-predicts the p90 size ratio, 36× against the

observed 59×. And the latency/race structure is not estimable from any public crypto feed — measuring races requires failed-order message data of the kind Aquilina, Budish, and O'Neill (2022) obtained from a regulator — so `hft_latency_us` and the race remain swept design parameters throughout. The calibrated claim is therefore deliberately narrow: the *environment* the agents trade in (arrival intensity, fundamental volatility) is disciplined by data; the strategic structure is designed, not estimated.

3.8 Measuring the snipe-to-depth ratio on the tape

§4.7 sweeps the snipe-to-quote size ratio, and where the real market sits on that axis is what turns the surface from a parameter study into a claim. No public feed reports the quantity, so we measure it (`calibration/trade_book_join.py`).

The sample is 86 complete book-and-trade hours of BTCUSDT-perp — hours whose book file carries at least 30,000 snapshot rows against a nominal 36,000 at the collector's 10 Hz cadence, and for which a matching trade file exists. Completeness is the binding requirement because every trade must be divided by the top-of-book depth it actually met, read from the snapshot immediately preceding it rather than the one following, which already reflects the trade's own consumption. That yields 2.27 million taker orders, after regrouping Binance's per-price-level `aggTrade` records back into single taker orders — a correction without which multi-level sweeps, precisely the race flow of interest, fragment and bias the ratio downward.

Nothing in a public tape flags a snipe, so we do not label them: we report the distribution unconditionally and under a *race proxy* — an order arriving within 0.2 s of a snapshot on which the top of book moved by at least one price increment — and treat the gap between the two as the informative object. Conditioning on a price move rather than on order size is deliberate, since a move is what the BCS mechanism says triggers a race whereas conditioning on size would assume the conclusion. A placebo on non-move orders in the same window quantifies the dilution that 100 ms snapshots impose on a microsecond phenomenon: its median is 0.0057 against the proxy's 0.0283, a factor of 5.0, which is what licenses calling the proxy a bound rather than a measurement.

sample	n	p10	p50	p90	p99
unconditional	2,269,256	0.00027	**0.0072**	0.393	18.0
race proxy	390,540	0.00051	**0.0283**	4.24	290
placebo (no move)	1,849,504	0.00024	0.0057	0.208	2.18

The distribution is extremely right-skewed, which is why the surface sweeps as far as ratio 1.0. On the collector's current 105-hour sample the unconditional quartiles are p25 0.0011 and p75 0.0493 around a median of 0.0068, with the race-proxy median stable to 0.3%, so nothing in §4.7 turns on the sample boundary. Full alignment rules, the sweep de-fragmentation and aggressor-attribution corrections, and the placebo construction are documented in the replication archive.
4. Results

We report four completed experiments, all run through the same model-checked matching engine: continuous-market welfare decomposition (4.1), latency-advantage scaling (4.2), the HFT competition arms race (4.3), and batch auctions as a remedy (4.4). A preliminary Hawkes-bridge endogeneity probe did not reach significance and is not a main-paper result; we retain it only as supplementary scaffolding for forthcoming work, clearly demarcated in §4.5 below. The cross-cutting validation throughout is the zero-sum conservation residual, which closes to floating-point noise on every run, so that each welfare decomposition is an audited accounting identity rather than a modelling assumption.

4.1 Experiment 1: Welfare decomposition and the zero-sum conservation check

We begin by establishing that the welfare accounting in our setting is an audited identity rather than a modelling assumption. By construction, every unit of profit-and-loss in the model-checked matching engine accrues to one of three agent classes — high-frequency traders (HFT), market makers (MM), or noise traders (NT) — so the sum of realised PnL across classes must equal zero on every run. We measure the empirical conservation residual across 100 seeds at the operating point ($n_{\text{hft}} = 3$, adverse sensitivity 0.1, spread decay 0.1, MM latency $3000\,\mu s$). In the treatment economy the residual closes to machine zero, 2.78e-12 [-3.34e-12, 9.11e-12], and the matched no-HFT baseline behaves identically, -5.39e-12 [-2.16e-11, 1.05e-11]. Because the residual is indistinguishable from floating-point noise in both arms, the decomposition of welfare into HFT rent, MM loss, and NT loss is a verified accounting identity, and the rent magnitudes reported below are not subject to unmodelled leakage.

Within this conserved frame, HFT entry extracts a positive rent of 34.70 [26.25, 44.80] per run. The transfer is borne almost entirely by market makers: MM PnL changes by -28.50 [-38.31, -20.35] relative to the matched baseline (from 101.27 [93.28, 110.72] to 72.78 [71.05, 74.38]), while the noise-trader class absorbs a much smaller share, with an NT PnL delta of -6.21 [-7.87, -4.61]. At $n = 100$ this NT delta is now statistically distinguishable from zero — at $n = 20$ its interval straddled zero — but it remains an order of magnitude smaller than the MM delta and accounts for roughly 18% of the transfer. The welfare transfer is therefore predominantly, though not exclusively, borne by liquidity suppliers, consistent with HFT rent arising chiefly from picking off stale market-maker quotes rather than from worsened execution for uninformed flow.

The microstructure correlates of this transfer are pronounced. Liquidity gaps more than triple under HFT entry, 9.70 [9.15, 10.27] in treatment against 2.84 [2.47, 3.26] in the matched no-HFT baseline, a delta of 6.86 [6.20, 7.52]. Lagged Kyle's λ, our measure of price impact per unit signed flow, rises by roughly an order of magnitude, from 0.200 [0.184, 0.216] to 1.372 [1.361, 1.383]; the coefficient takes its name and its interpretation from Kyle (1985), where λ is the inverse of market depth in a continuous-auction model with a strategic informed trader, so a rise in it is properly read as the book becoming easier to move per unit of signed order flow. Table 1 collects these estimates.

Table 1. Experiment 1 welfare and microstructure estimates at the operating point ($n = 100$ seeds; point estimate with 95% bootstrap CI).

Quantity	Treatment	No-HFT baseline	Treatment − baseline
HFT rent	34.70 [26.25, 44.80]	—	—
MM PnL	72.78 [71.05, 74.38]	101.27 [93.28, 110.72]	-28.50 [-38.31, -20.35]
NT PnL	-107.48 [-116.84, -99.26]	-101.27 [-110.72, -93.28]	-6.21 [-7.87, -4.61]
Zero-sum residual	2.78e-12 [-3.34e-12, 9.11e-12]	-5.39e-12 [-2.16e-11, 1.05e-11]	—
Liquidity gaps	9.70 [9.15, 10.27]	2.84 [2.47, 3.26]	6.86 [6.20, 7.52]
Lagged Kyle's λ	1.372 [1.361, 1.383]	0.200 [0.184, 0.216]	—

We stress-test the rent result against the two parameters that most directly govern the adverse-selection channel, sweeping adverse sensitivity and spread decay over the grid $\{0.05, 0.1, 0.2\} \times \{0.05, 0.1, 0.2\}$ with $n = 100$ seeds per cell. HFT rent is significantly greater than zero in all nine cells, and the zero-sum identity holds in all nine, with point estimates of the rent ranging from 16.56 to 46.63. The rent is thus a robust feature of the calibration rather than an artefact of a single operating point.

Two points of nuance bear on how the result should be read. First, the no-HFT baseline is mildly fragile: liquidity gaps in that arm are not exactly zero (2.84 [2.47, 3.26]), reflecting residual quote churn even absent HFT participation, so the baseline does not represent a frictionless market. We nonetheless interpret the treatment-versus-baseline contrast as informative because the gap CIs are non-overlapping (treatment 9.70 [9.15, 10.27] versus baseline 2.84 [2.47, 3.26]). Second, the NT PnL delta is small but, at $n = 100$, statistically distinguishable from zero (-6.21 [-7.87, -4.61]); we therefore state that HFT entry imposes a measurable but secondary cost on noise traders, roughly 18% of the total transfer against the market maker's 82%. The welfare story of Experiment 1 remains a predominantly market-maker-to-HFT transfer occurring under a conserved, machine-zero accounting identity, accompanied by a sharp deterioration in measured liquidity.

4.2 Experiment 2: Latency-advantage scaling

Experiment 2 sweeps the market maker's staleness — the latency with which it observes the fundamental — holding the HFTs at zero latency, and asks how the transfer scales. The question matters for the incidence surface: a boundary mapped across a mechanism containing a discontinuity would be an artifact of the discontinuity rather than a boundary.

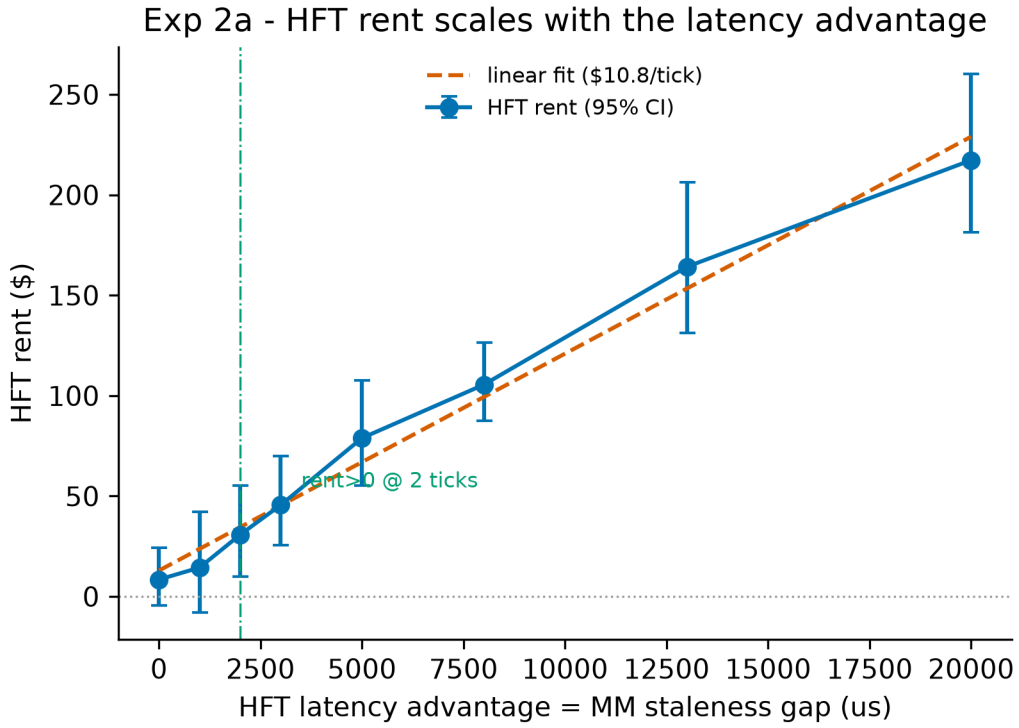


Figure 1: Figure 2a: HFT rent versus MM staleness with linear-fit overlay and significance threshold

Rent rises with staleness at approximately $10.46 per tick, and the relationship is smooth: comparing a linear specification against a segmented one with a free breakpoint gives $\Delta\text{BIC} = -0.17$, a magnitude well inside the range of indifference, so the data do not discriminate between them. We report this as the absence of a detectable knee rather than as positive support for linearity — a distinction that matters, because at our sample size an absence of evidence for a threshold is not evidence of its absence. Rent first becomes statistically distinguishable from zero at roughly two ticks of staleness, which is the detectability floor of this grid rather than a property of the mechanism, since latency resolves only at tick granularity here (§5.2).

The welfare decomposition closes at every point on the sweep, and liquidity gaps and HFT fill rate move with staleness in the directions the mechanism predicts: a staler maker is picked off more often, and its thinned book produces more one-sided episodes. Per-cell values across the sweep are in the replication archive; the figure carries the shape, which is what the incidence argument uses. ### 4.3 Experiment 3: HFT competition and the socially wasteful arms race

We next ask how the number of competing high-frequency traders, k, shapes the rents they extract and the costs they impose. We sweep k over the grid $\{0, 1, 2, 3, 5, 8, 13, 21\}$ with $n = 100$ seeds per cell, holding the market-maker (MM) population and order-flow process fixed. The central finding is a sharp separation between the *private* and *social* consequences of entry: individual HFT profitability collapses as competitors enter, yet the aggregate rent transferred away from liquidity providers does not. The marginal social cost of an additional HFT is therefore not a larger wealth transfer but a more fragile book.

Per-HFT rent falls monotonically and steeply, from \$34.82 [26.40, 44.84] at $k = 1$ to \$1.60 [1.21, 2.07] at $k = 21$ (Figure 3a). The decline is steep and close to hyperbolic, and the same structure appears in execution shares: the per-HFT fill rate falls from 0.2886 [0.2863, 0.2909] at $k = 1$ to 0.0139 [0.0137, 0.0140] at $k = 21$ (Figure 3c). Competition thus partitions a fixed pool of exploitable flow into ever-thinner per-agent slices without enlarging the pool itself.

The aggregate confirms this directly. Total extracted rent is flat across the entire grid (Figure 3, Figure 3b): \$34.82 [26.40, 44.84] at $k = 1$, \$34.93 [26.55, 44.87] at $k = 2$, \$34.70 [26.25, 44.80] at $k = 3$, \$33.83 [25.36, 44.03] at $k = 5$, \$34.50 [25.84, 44.76] at $k = 8$, \$34.43 [25.63, 44.87] at $k = 13$, and \$33.70 [25.39, 43.45] at $k = 21$ — a derived mean of \$34.42 (std 0.48, range 33.70–34.93). The $n = 100$ estimates make this flatness materially sharper than our earlier $n = 20$ run, which spread over 43.64–47.21 (std 1.11); the spread across the grid is now roughly a third as wide, so the constancy of total rent is a stronger claim than before, not merely a rescaled one. The transfer is cleanly zero-sum: MM PnL is approximately constant near \$73 for all $k \geq 1$ and the zero-sum residual is numerically zero at every count. We therefore decline to characterize HFT entry as raising total welfare loss; the welfare delta tracks total rent and is flat, not rising.

k	Per-HFT rent (\$)	Total rent (\$)	Fill rate	Liquidity gaps
0	—	—	—	2.84 [2.47, 3.26]
1	34.82 [26.40, 44.84]	34.82 [26.40, 44.84]	0.2886 [0.2863, 0.2909]	5.27 [4.91, 5.65]
2	17.46 [13.28, 22.43]	34.93 [26.55, 44.87]	0.1448 [0.1437, 0.1460]	7.79 [7.31, 8.29]
3	11.57 [8.75, 14.93]	34.70 [26.25, 44.80]	0.0964 [0.0957, 0.0971]	9.70 [9.15, 10.27]
5	6.77 [5.07, 8.81]	33.83 [25.36, 44.03]	0.0578 [0.0574, 0.0583]	14.92 [14.27, 15.61]
8	4.31 [3.23, 5.60]	34.50 [25.84, 44.76]	0.0362 [0.0359, 0.0365]	21.09 [20.20, 21.99]
13	2.65 [1.97, 3.45]	34.43 [25.63, 44.87]	0.0223 [0.0222, 0.0225]	32.79 [31.73, 33.91]
21	1.60 [1.21, 2.07]	33.70 [25.39, 43.45]	0.0139 [0.0137, 0.0140]	47.47 [46.22, 48.85]

Where the marginal cost of entry does appear is in market fragility. The incidence of liquidity gaps rises steadily with k, from a baseline of 2.84 [2.47, 3.26] at $k = 0$ to 47.47 [46.22, 48.85] at $k = 21$ — roughly a 16.7-fold increase (Figure 3b). The growth is well described as linear in k

(derived fit: slope = 2.13, intercept = 3.74, $R^2 = 0.99730$). This is the substantive welfare result of the experiment: each additional HFT competes away its predecessors' per-agent rent without enlarging the total transfer, but contributes an approximately constant increment to the rate at which the book is left empty. The arms race is socially wasteful not because it grows the rent pie but because the competition to capture a fixed pie progressively thins the liquidity available to ordinary participants.

4.4 Experiment 4: Batch auctions as a remedy

Experiment 4 switches the clearing rule from continuous matching to a uniform-price call auction, holding every agent and market parameter fixed, and sweeps the clearing interval. Both grids clear through the verified engine's own `uncross` (§3.2), so the comparison isolates the clearing rule rather than a reimplementation of it.

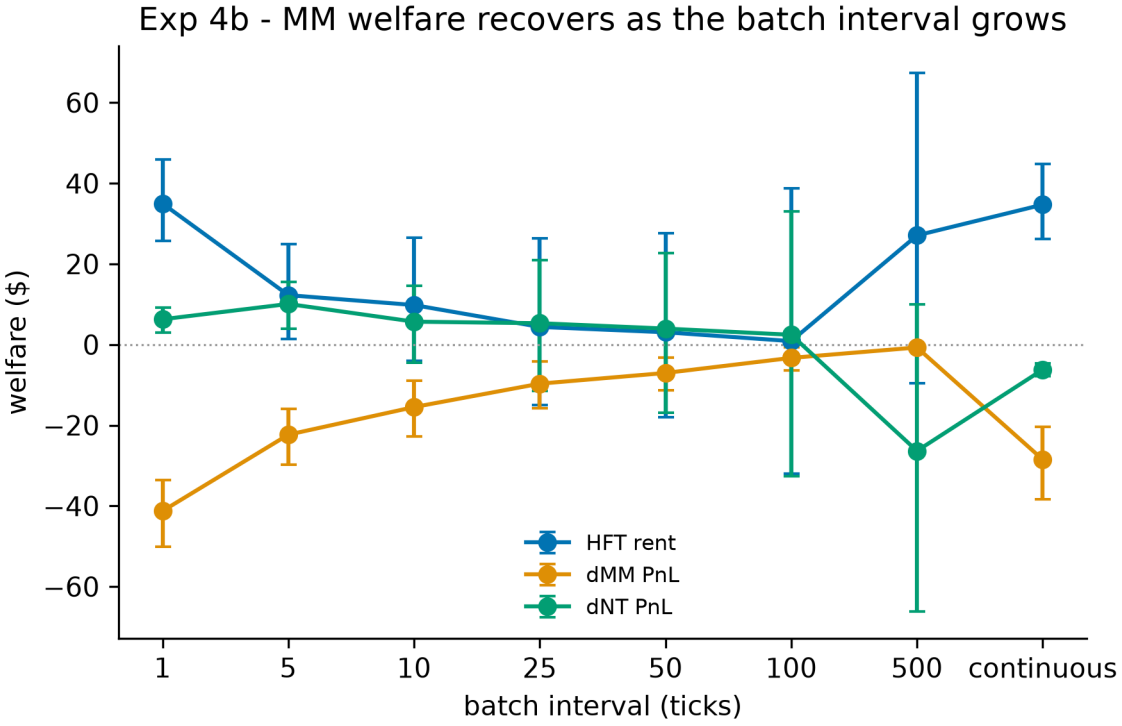


Figure 2: Welfare decomposition versus clearing interval

Market-maker welfare recovers monotonically once the interval is long enough for the slow quoter to participate: at-least-50% recovery by an interval of about 25 ticks, and about 97% by 500, taking maker loss from \$28.5 to \$0.7. Over 50,000-tick runs every interval ≥ 5 recovers effectively all of the loss. This MM-recovery result is the robust signal in the experiment, resting on a matched-seed decomposition rather than on the rent estimator.

The accompanying reduction in HFT rent points the same way but is not established. At 3,000 ticks only the interval-5 cell's rent interval excludes zero, and at 50,000 ticks the estimator's dispersion — which grows super-linearly in run length — swamps every batch cell, so we draw no rent conclusion from the long grid and decline to nominate an optimal interval on rent grounds. At intervals above 100 ticks those cells are uninformative and are reported in the paper's replication archive.

The interval-1 cell is the experiment's other finding and runs the other way. A per-tick auction does not merely fail to help: maker loss worsens to \$41.3, against \$28.5 under continuous clearing,

and liquidity gaps spike to about 137 against a continuous baseline of 9.7. A maker that cannot refresh its quotes within a single clearing cadence is churned by the auction and repriced against before it can react. Batching is therefore a remedy only above a minimum cadence, and that cadence is itself a design parameter, to be matched to the speed at which liquidity providers can revise. ### 4.5 The Hawkes endogeneity diagnostic: a placebo-validated null

Experiment 5 asks whether the liquidity gaps of Experiments 1–3 are preceded by rising order-flow self-excitation — whether they are reflexive cascades rather than direct consequences of adverse-selection widening. We estimate the branching ratio of a self-exciting Hawkes process on simulated order flow (Filimonov and Sornette 2012) in windows labelled pre-gap or normal, on both the trade and quote-update processes, with a placebo arm whose gap labels carry no information. The windowing, labelling, and placebo construction are documented in the replication archive.

Arm / process	pre mean $n(t)$	normal mean $n(t)$	p (one-sided)	rank-biserial	n_{pre} / n_{normal}
Treatment / trades	0.0162	0.0167	0.258	0.012	1,416 / 3,614
Treatment / quotes	≈ 0	≈ 0	0.937	-0.029	1,260 / 3,206
Placebo / trades	0.0111	0.0168	0.895	-0.038	373 / 8,614
Placebo / quotes	≈ 0	≈ 0	0.491	0.001	335 / 7,707

The result is a null, and the placebo confirms the pipeline does not manufacture endogeneity where none exists. Pre-gap and normal branching ratios are indistinguishable on trade arrivals ($p = 0.258$, effect size 0.012), and the quote-update process shows essentially zero self-excitation anywhere — mechanically sensible, since the maker's requotes track an exogenous random walk. At this operating point the gaps are therefore not preceded by rising self-excitation: they arise from the adverse-selection widening mechanism directly. This is a negative result about the simulated mechanism and is stated as such. It is not evidence against reflexivity in real markets, where self-excitation is well documented; this market's arrival processes are close to Poisson by construction, with branching ≈ 0.017 on trades, some $50\times$ below criticality, leaving little endogeneity to detect. What the experiment contributes is a validated instrument, ready for a real-data leg that awaits a depth-depletion gap definition suited to a book that never empties (§5.3). No figure accompanies this result. ### 4.6 Experiments 1–4 at calibrated parameters

The pre-registered operating point was chosen for mechanism clarity, not realism. §3.7 fits arrival intensity and fundamental volatility to BTCUSDT-perp moments, and this section re-runs Experiments 1–4 at those fitted values with a lognormal size model and the perp's book depth (`run_calibrated.py`). Every qualitative finding survives the move: the transfer remains a machine-precision zero-sum, rent scales smoothly in maker staleness with no phase transition, aggregate rent stays flat while fragility rises in HFT count, and batch auctions recover maker welfare above a minimum cadence while the per-tick auction backfires. That survival is the strongest evidence we can offer that the findings are properties of the clearing design rather than of a convenient parameterisation.

What calibration adds is magnitude. Normalised by traded notional, HFT rent moves from 0.125 bp at the operating point to 4.42 bp calibrated. The calibrated figure is an upper bound rather than an estimate, because a single latency-disadvantaged maker facing the perp's order flow has no competitor to replenish a cleared quote and so absorbs the entire race. Replacing it

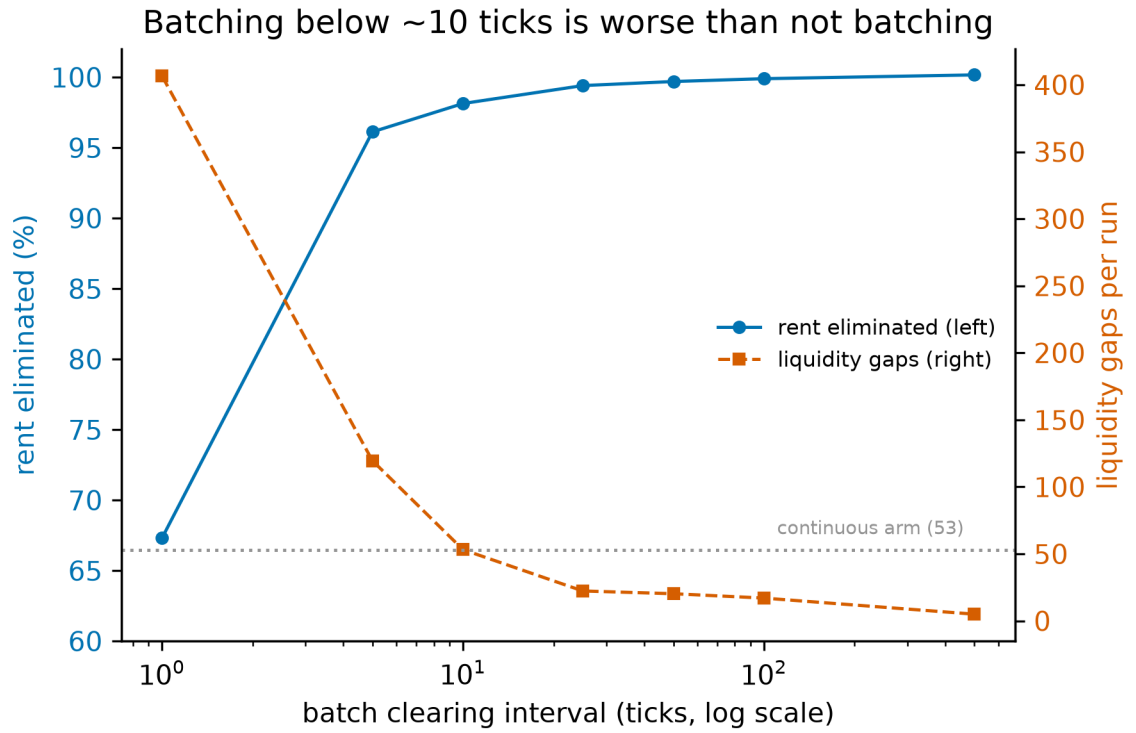
with N makers whose latencies span the measured top-of-book replenishment quantiles, holding aggregate depth fixed, collapses the rent by a factor of fifty:

N makers	latencies (ticks)	HFT rent (bp)	liquidity gaps
1	3	4.421	52.73
2	1, 5	0.801	38.00
3	1, 3, 5	0.324	16.29
5	1–5	0.089	2.71

$n = 100$ seeds; the $N = 1$ row reproduces the published single-maker figure, so the sweep nests it rather than re-specifying it, and the zero-sum residual stays at machine zero throughout. The $\approx$0.4 bp latency-arbitrage tax that Aquilina, Budish, and O'Neill (2022) measure from regulator-held exchange messages falls inside the range these arms span — envelope consistency between a simulated and a directly measured quantity, not agreement on a number.

The mechanism is visible in the per-maker decomposition. At $N = 5$ maker PnL is monotone in latency, the fastest earning +2,014 and the slowest losing −2,393, because a fast competitor replenishes the touch before the slow maker's stale quote can be sniped. The slow maker still absorbs adverse selection; what changes is that it is no longer the only supplier. The single-maker absorption bias is therefore measured and removed rather than asserted as a caveat.

Incidence is not pinned by the data. The main arm sets `hft_qty` = `quote_qty`, preserving the operating point's ratio of 1.0 so a snipe clears the top quote. Holding `hft_qty` at 50 instead — a snipe taking 2.4% of the book — changes not the size of the effect but its direction: total rent rises in k rather than staying flat, per-HFT rent flattens instead of decaying as $1/k$, and the maker's PnL delta is *positive* for $k \leq 5$, turning negative only at $k \geq 8$, with noise traders bearing the loss throughout. The claim that rent is extracted from the market maker is therefore conditional on the snipe-to-quote size ratio. Two ratio points do not locate a boundary in a two-dimensional space, which is what §4.7 maps. At calibrated parameters the batch arm also remains non-monotone in the clearing interval, the interval-1 cell still the worst performer on both rent elimination and gaps.



4.7 The incidence surface: who bears the latency tax

§4.6 established that the *incidence* of the transfer moves with the snipe-to-quote size ratio, on the evidence of two ratio points. Two points and one k-sweep cannot locate a boundary in a two-dimensional space, and the direction of the effect — the maker profiting from an arms race conducted against it — is surprising enough to deserve a mapped region rather than a sensitivity note. This section maps it. We sweep snipe size against HFT count on the calibrated environment of §3.7, seven snipe quantities × nine HFT counts at n=100 seeds (6,400 engine runs), holding the maker count at one (`experiments/exp6_incidence_surface.py`, `exp6_incidence_surface.json`).

Validity: the surface nests the published cells rather than re-specifying the environment. Because a new grid could differ from §§4.1–4.6 for uninteresting reasons, the ratio axis is swept as integer snipe *quantities* so that two cells coincide exactly with results already reported. The ratio-1.0 column reproduces `exp3_hft_count_calibrated.json` to every quoted digit — rent 85,721 / 85,163 / 84,375 / 83,262 / 81,469 / 78,696 / 74,261 and liquidity gaps 32.05 / 42.19 / 52.73 / 74.19 / 106.51 / 151.08 / 217.78 across k = 1 … 21 — and the ratio-0.0243 row reproduces §4.6's robustness arm to the digit, including the maker's +1,327 delta at k=1 and −5,213 at k=21. The zero-sum residual is machine zero in all 63 cells (largest 5.6×10^−10 against rents of order $10^{4–10}$5), so the conservation identity that audits every other result in this paper audits this one unchanged.

The boundary. Incidence is classified per cell by the sign of the maker's PnL delta against the shared no-HFT baseline, at 95% bootstrap confidence; a cell whose interval brackets zero is reported as indeterminate rather than assigned to whichever side its point estimate falls on.

Snipe/quote ratio	Maker gains up to	Flips at	Maker delta at k=21
0.0049	all k ≤ 21	—	+2,154 (still gains)
0.0097	all k ≤ 21	—	+1,425 (still gains)
0.0243	k = 6	k = 7	−5,213

Snipe/quote ratio	Maker gains up to	Flips at	Maker delta at k=21
0.0485	k = 2	k = 3	−17,203
0.1213	k = 1	k = 2	−39,006
0.3396	none	below k=1	−61,978
1.0000	none	below k=1	−72,332

At the race proxy reading of 0.0283, the flip occurs at k=7. If the real market operates with more than seven competing snipers at that ratio, the inversion does not hold under the race proxy.

The critical count falls monotonically as the snipe grows, from beyond the grid at the smallest ratios to below one at the largest — a collapse spanning the full range of k over roughly one order of magnitude in ratio. At the two smallest ratios no flip occurs anywhere on the grid: the maker's gain does not merely survive competition, it *increases* in k (from +1,061 at k=1 to +2,154 at k=21 at ratio 0.0049). At the two largest the maker loses even to a single sniper. §4.6's two points were one slice each of this surface.

Where the real tape sits, and what it does to the BCS incidence prediction. The ratio is a design parameter in the sense that no feed reports it directly, but §3 bounds it: measured per trade against contemporaneous top-of-book depth on 86 complete book-and-trade hours of BTCUSDT-perp, it is 0.0072 at the median unconditionally and 0.0283 under the race proxy, with the placebo quantifying the dilution. The unconditional median falls between the two smallest ratios on the grid — and that is the region in which the maker gains at every HFT count we tested, while noise traders bear the entire transfer. **In the regime the tape supports, under single-maker liquidity supply, the BCS prediction that the latency tax falls on liquidity providers inverts.** That qualifier is load-bearing and is discharged in §4.8, which adds the maker-count axis and finds the inversion does not survive a second maker; this subsection's conclusion should not be read on its own. The maker widens in response to adverse fills, and that widening leaves it better off on net — by cutting what the snipers take from its stale quotes rather than by earning more from noise flow, which in fact falls (see the leg decomposition below); the loss lands on the noise traders who cross the wider spread. The race-proxy median of 0.0283 sits just above the 0.0243 row, where the flip occurs at k=7 — so under the more conservative conditioning the maker still gains against a handful of snipers and loses only once the field grows. Both readings place real BTC at or below the boundary rather than deep in the maker-pays regime that §4.1 reports, and the maker-pays regime is reached only at snipe sizes 35–140× the measured median. One qualification attaches to the measurement: the tape under-measures multi-level sweeps, because Binance aggregates fills per taker order per price level, so the true race-order ratio is somewhere above the proxy — which is precisely why the boundary matters more than any single point on it.

The boundary is plotted with its censoring made explicit, because only three of the seven ratios flip inside the grid: at 0.0049 and 0.0097 the maker gains at every count up to twenty-one, so the critical count is bounded below by 21 rather than measured, and at 0.3396 and 1.0000 the maker already loses to a single sniper, so it is bounded above by 1. Drawing a curve through all seven would fabricate four of them. The figure's substantive content is the position of the tape band relative to the boundary: the band lies entirely within the maker-gains region across the full range of counts we swept, and for the BCS incidence prediction to hold on this market the measured ratio would have to sit above the boundary curve rather than below it.

A secondary regularity: the noise-trader toll is nearly invariant. Across all 63 cells the noise traders' delta stays within −1,699 to −3,410, a factor of two, while the maker's delta swings from +2,154 to −84,023, a factor of forty in magnitude and a change of sign. The

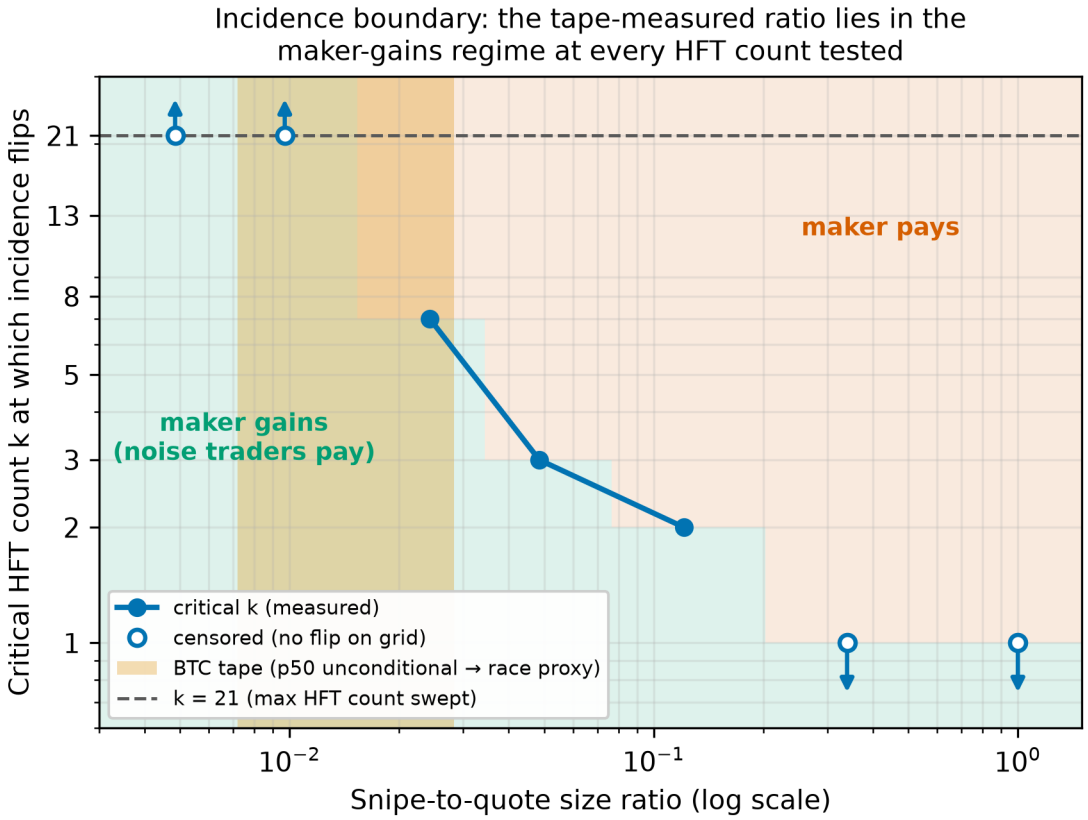


Figure 3: Incidence boundary over snipe-to-quote ratio and HFT count. The shaded vertical band is the ratio measured on the BTCUSDT-perp tape (unconditional median to race-proxy median); open markers with arrows are ratios at which no flip occurs anywhere on the swept grid.

maker's exposure is therefore essentially the whole source of variation on this surface. Noise traders pay a roughly fixed toll for transacting against a latency-disadvantaged maker in the presence of snipers, and what the ratio and the count determine is whether the maker adds to that toll or collects part of it. This is a cleaner statement of the welfare picture than a single-cell decomposition can give, and it survives the conservation check in every cell.

The near-invariance is what inelastic demand predicts. Noise-trader arrival in this section does not respond to the spread: each agent crosses with fixed probability `lambda_per_tick` whatever the maker is quoting (§3.3), so a near-constant toll across 63 cells is the signature that assumption leaves. Volume is pinned by construction, so the only channel through which widening can reach the noise traders is the price paid per crossing, and the factor-of-two range is that channel operating alone. We therefore cannot separate the reading in which the toll is invariant for economic reasons from the reading in which the model gives it no margin on which to vary. §4.8 supplies the missing margin by making arrival elastic in the quoted half-spread, and finds the incidence unmoved.

A negative result: the surface does not reduce to one dimension. The boundary invites an obvious simplification — that only aggregate snipe pressure matters, so incidence should turn on the product of ratio and count rather than on each separately. At the flips the product ratio × k lies in a narrow band, roughly 0.10–0.20, which makes the reduction look right. We tested it and it fails. At ratio 0.0097 with k=21 the product is 0.204 and the maker still gains significantly; at ratio 0.0243 with k=7 the product is 0.170 — smaller — and the maker no longer gains. A single product threshold cannot order those two cells, so the near-collapse is a coincidence of the grid rather than a law, and the structure is genuinely two-dimensional. We report no product rule. The mechanism is presumably that snipe size and sniper count act through different channels — size determines how much of the quote a single race removes, count determines how often the quote is stale when a race starts — and the maker's widening response is not linear in either.

Decomposing the inversion into its two legs. The explanation offered above — the maker's widening earns more from noise flow than snipers take from its stale quotes — is a story about a difference of two quantities, and the surface reports only the difference. We therefore measured the legs separately at the tape-band cells (`experiments/exp7c_leg_decomposition.py`, $n = 100$, single maker as in the rest of this section). Every maker fill is scored as edge against the fundamental at fill time, positive when the maker captured value, and partitioned by the *same* adverse test that triggers the widening (§3.3), so the decomposition is measured with the mechanism's own discriminator rather than an external proxy. Because `mm_pnl` is mark-to-market while the legs are realized edge at fill time, inventory carry is reported as an explicit third column rather than absorbed into either leg; the three sum to the net delta exactly, and the net column reproduces the surface's ΔMM (largest reconciliation error 3.6×10^{-12}).

ratio	k	revenue leg	sniping leg	inventory carry	net ΔMM
0.0049	1	−2,949	+2,289	+1,721	**+1,061**
0.0049	3	−5,481	+4,787	+2,282	**+1,588**
0.0049	7	−6,487	+6,040	+2,360	**+1,913**
0.0049	21	−6,747	+6,526	+2,374	**+2,154**
0.0097	1	−4,329	+3,512	+2,062	**+1,244**
0.0097	3	−6,249	+5,448	+2,343	**+1,542**
0.0097	7	−6,852	+6,041	+2,369	**+1,558**
0.0097	21	−6,529	+5,566	+2,389	**+1,425**

Seed-matched deltas against the shared no-HFT baseline, $n = 100$.

The table does not support the mechanism as stated, and the correction matters. The revenue leg is **negative** in all eight cells: with snipers present the maker earns *less* from non-adverse flow than it does in the no-HFT baseline, because widening drives away more noise volume than it recovers in edge per fill. The sniping leg is **positive**, meaning the maker's losses to adverse fills *shrink* relative to baseline — the wider quote is picked off less profitably, and the baseline itself carries adverse selection from noise traders hitting stale quotes when the fundamental moves. The net gain is therefore not "more noise revenue than sniping loss." It is reduced adverse-selection cost, plus a substantial inventory-carry term, together outweighing a fall in noise revenue.

Two consequences follow. The inversion is real and is not an artifact — the net column is the surface's own ΔMM and it is positive throughout — but the intuition attached to it in this section, and repeated in §1 and §6, is the wrong way round on the noise-revenue leg, and we flag it as such rather than restating the table to fit it. And the inventory-carry column is large enough (roughly a third to a half of the gross legs' difference) that a decomposition omitting it would misattribute the source of the gain; the maker's benefit is partly that it holds inventory through a favourable mark, which is a distinct channel from either leg and one the widening story does not describe at all. Establishing which of the three channels dominates under elastic demand is the first item of §5.3.

The claim holds for all HFT counts tested — up to twenty-one. Behavior at higher counts is not measured.

4.8 Structural robustness: maker competition and elastic demand

§4.7 holds two things fixed that its own limits identify as load-bearing: liquidity is supplied by a single maker, and uninformed demand is inelastic. This section relaxes each in turn. The maker-count axis reverses the inversion at the class level; elasticity does not move it. Both extensions nest §4.7 exactly at their inelastic, single-maker corner, checked numerically rather than asserted.

Maker competition. We re-run the 63-cell surface at maker counts $M \in \{1, 2, 3, 5\}$ on the calibrated environment, holding every §3.7 parameter fixed, at $n = 20$ (441 cells, `exp7_3d_surface.py`). Aggregate top-of-book depth is held at `quote_qty` and divided across the makers, so adding makers varies competition and latency spread rather than the depth a snipe meets; latencies for $M > 1$ are §4.6's set, anchored on measured replenishment quantiles. A `zero_fast` variant placing the leading maker at zero latency is run as an optimistic bound. The $M = 1$ column reproduces §4.7 to **0.0 absolute difference** on five metrics at three probe cells re-run at $n = 100$, and conservation holds across all 441 cells at a maximum absolute residual of 1.26×10^{-9}.

M	latencies (ticks)	ratio 0.0049	ratio 0.0097	flip @ 0.0243	rent at $k = 3$, ratio 1.0
1	3	**gains at all** $k \leq 21$	**gains at all** $k \leq 21$	$k = 6$	85,638
2	1, 5	gains only at $k = 1$	no gains at any k	none	22,442
3	1, 3, 5	no gains at any k	no gains at any k	none	9,541
5	1–5	no gains at any k	no gains at any k	none	2,548

The inversion fails at $M = 2$, the smallest competitive case. Of eighteen tape-band cells, eighteen are maker-gains at $M = 1$, one at $M = 2$, none at $M = 3$ or $M = 5$; the `zero_fast` arms show none at any $M > 1$. This is a sign change carried by the intervals rather than a loss of power: at $k = 3$, ratio 0.0097, the maker's delta runs +1405 [854, 2024] at $M = 1$, −707 [−1224, −114] at $M = 2$, −675 [−847, −493] at $M = 3$, −348 [−599, −115] at $M = 5$. The noise-trader toll reverses in step, from −2,669 under a single maker to −78 at five, so competitive supply reinstates the incidence BCS predicts rather than merely cancelling the inversion. Two channels drive it and both are real: splitting aggregate depth multiplies each maker's own snipe-to-quote ratio by M — the aggregation problem of §5.2 measured rather than argued — and, at matched per-maker ratio, competition still moves the boundary (at per-maker 0.0243 the single maker gains through $k = 5$ while $M = 5$ already pays from $k = 5$).

The aggregate conceals a within-class redistribution, and this is the finding that survives. A negative summed delta is consistent with every maker losing or with fast makers gaining while slow ones fund them. Re-running the tape-band cells at $M \in \{2, 3\}$ with each maker's delta bootstrapped separately, at $n = 100$ (72 cells, `exp7b_per_maker_pnl.py`, maximum residual 5.57×10^{-10}), every arm is redistributive. At $M = 2$ under `zero_fast` the leading maker gains in **all eighteen** band cells with its interval excluding zero — +2,284 [2,096, 2,477] at $k = 3$, ratio 0.0097 — while the five-tick maker pays in all eighteen, −3,475 [−3,656, −3,303]. There is no cell in which both lose. At $M = 3$ the two fastest gain and the slowest funds both. One arm is not monotone in latency and we report the anomaly rather than a mechanism for it: under calibrated $M = 3$ the *middle* maker is the sole consistent winner (+630 [529, 734]) while the fastest pays for $k \leq 7$. §4.6's monotone-in-latency ordering was measured at ratio 1.0 and does not extend to the tape band.

Elastic demand. The second fixed assumption is that noise traders cross at a fixed per-tick probability whatever the maker quotes. The objection this licensed — elasticity withdraws the flow that finances the maker's gain — rests on a premise §4.7's leg decomposition already refuted: the revenue leg is *already negative*, and the gain comes from the sniping leg shrinking. Elasticity therefore deepens a leg already working against the maker without touching the dominant channel, making the question quantitative. We set $\lambda_{\text{eff}} = \lambda(s_{\text{base}}/s)^{\alpha}$ with s read off the book, so arrivals match §3.7's fitted intensity at base spread for every α and only the response to widening changes, and re-run the full surface at $\alpha \in \{0, 0.5, 1.0\}$ (189 cells, `exp8_elastic_demand.py`). The $\alpha = 0$ path short-circuits before the book is read, preserving the RNG stream: it reproduces §4.7 to 0.0 absolute difference on `hft_rent` and `zero_sum_residual` at all three probes, with maximum residual 1.14×10^{-8} and baseline trades falling 5,165 → 4,902 → 4,657, confirming the knob engages.

α	ratio 0.0049	ratio 0.0097	flip @ 0.0243	rent at $k = 3$, ratio 1.0	baseline trades
0.0 (inelastic)	gains at all $k \leq 21$	gains at all $k \leq 21$	$k = 6$	85,638	5,165
0.5 (mild)	gains at all $k \leq 21$	gains to $k = 13$	$k = 5$	86,931	4,902
1.0 (unit)	gains at all $k \leq 21$	gains to $k = 13$	$k = 5$	88,502	4,657

The maker-gains region survives. Eighteen of eighteen tape-band cells gain inelastically and **seventeen at both mild and unit elasticity**; the single loss, ratio 0.0097 at $k = 21$, turns *indeterminate* rather than maker-pays (+215 [−202, 672] at $\alpha = 1.0$), and no band cell classifies as maker-pays at any elasticity tested. The lower band edge is untouched, the 0.0243 boundary retreats one grid step, and the effect saturates between $\alpha = 0.5$ and $\alpha = 1.0$ rather than scaling.

This is an out-of-sample test of the mechanism correction rather than a restatement of it: on a revenue-capture account unit elasticity should have been close to fatal, since it removes a tenth of noise participation exactly when the maker widens. Noise traders do withdraw as the objection predicted — their loss shrinks from −2,667 to −2,033 — which buys them partial self-protection but does not move the incidence.

What the two extensions jointly establish. The class-level answer is structure-dependent and the within-class answer is not. Whether summed maker PnL is positive depends on how many makers supply the book; that fast makers gain and slow makers pay does not, and it survives elasticity to unit. Both surfaces are mapped at $n = 20$ while the within-class decomposition is at $n = 100$, so the class-level reversal, though carried by intervals excluding zero, is not powered to the standard the within-class result meets. ## 5. Discussion

The four completed experiments describe a consistent picture of latency competition on a continuous limit order book whose matching layer is model-checked (the Hawkes diagnostic of §4.5 adds a placebo-validated null: the gaps are not preceded by rising order-flow self-excitation at this operating point). The arms race is a transfer rather than a destruction of value (§4.1): HFT rent of \$34.70 [26.25, 44.80] is borne predominantly by the market maker (MM PnL delta \$-28.50 [-38.31, -20.35]), with a smaller but measurable noise-trader delta (\$-6.21 [-7.87, -4.61]), all under a conservation residual indistinguishable from floating-point noise (mean 2.8×10^{-12}). That transfer scales smoothly in market-maker staleness, and we find no evidence of a critical threshold (§4.2; slope \$10.460 per tick, $\Delta\text{BIC} = -0.17$ — a magnitude too small to discriminate between the linear and segmented specifications, so the finding is the absence of a detectable knee rather than positive support for linearity). As competitors multiply, per-HFT rent dissipates as roughly $1/k$ while the *total* extracted rent stays flat near \$34.4, so the marginal social cost of an additional HFT surfaces not as a larger wealth transfer but as a more fragile book (§4.3; gaps rising approximately linearly, $\approx 2.1k + 3.7$, about 17× by twenty-one HFTs). Batch auctions recover market-maker welfare monotonically away from the degenerate single-tick corner (§4.4), though their effect on HFT rent remains statistically unresolved at long clearing intervals. Below we draw out what these findings do and do not license, state their limitations plainly, and set out the work needed to harden them.

5.1 Implications for market design

Three findings carry policy weight, and each is narrower than the headline it replaces.

First, the class-level answer to "who bears the latency tax" is structure-dependent, so a regulator cannot read it off the existence of the tax. Whether summed maker PnL is positive or negative turns on how many makers supply the book and on the snipe-to-depth ratio prevailing there — quantities that differ across venues and that no public feed reports — so an incidence claim transported from one market to another is an assumption rather than a measurement.

Second, within-class redistribution survives both structural tests and is the more robust object. Fast makers gain at the tape-measured ratio while slow ones fund both them and the snipers, and that pattern holds under competitive supply and under demand elastic to unit. The policy reading is that latency competition sorts liquidity providers rather than taxing them uniformly: a remedy justified as protecting "liquidity providers" protects the slow ones and, at the measured ratio, takes from the fast.

Third, the clearest design warning is the interval-1 backfire. A per-tick auction does not merely fail to help — it raises maker loss to \$41.3 against the continuous \$28.5 and spikes liquidity gaps to about 137 against 9.7, because a maker that cannot refresh within one clearing cadence is churned and repriced against before it can react. Batching is a remedy only above a minimum cadence matched to the speed at which liquidity providers revise quotes, and a venue setting the

interval too aggressively reproduces the pathology it meant to cure. That the recovery signal is monotone above that floor while the accompanying rent reduction is not statistically resolved (§4.4) is itself the reason to frame the remedy in terms of who is protected rather than of the sniper's take. ### 5.2 Limitations

Several limitations bound the strength of these conclusions.

First, the elasticity of uninformed demand is the assumption the incidence result leans on hardest, even now that it is measured. Noise traders cross at a fixed per-tick probability in every arm through the competition surface (§3.3), so the volume a maker earns its widened spread on cannot fall when it widens. §4.8 relaxes this to unit elasticity and the maker-gains region very largely survives: eighteen of eighteen tape-band cells gain inelastically and seventeen at both mild and unit elasticity, the single loss turning indeterminate rather than maker-pays, with the 0.0243 boundary retreating one grid step and the lower band edge untouched. That resolves the limit in the direction §4.7's leg decomposition predicts — elasticity deepens an already-negative revenue leg without touching the loss-reduction channel that produces the gain — but it does not eliminate it. Elasticity above unit is untested, as is its interaction with the maker-count axis, where the surviving gain belongs to the fast maker and runs through the same channel.

Second, liquidity supply is narrow in both count and rule. §4.8 maps maker counts to five, but §4.7's headline arm holds one maker, and every maker in this paper follows a single imposed adverse-selection quoting rule. A maker that defended itself differently — fading quotes, skewing on inventory — might not profit from small snipes at all, and none of ours chooses its spread to maximise profit, so "the maker gains" is a property of that rule under sniping rather than a statement about profit-maximising liquidity provision. The rule's aggressiveness is one hand-tuned parameter, and re-running §4.7's load-bearing readings across a 7.5× span of it leaves the inversion's *existence* intact at the tape band while moving its *location* across the whole grid and changing total rent by a factor of five (`exp6b_sensitivity_adverse.py`); no critical HFT count should therefore be read as a property of the market rather than of this maker. Scope is narrow in ways we note rather than enumerate separately: one asset on one venue with no cross-venue routing or multi-asset correlation; spread and depth *levels* far from the perp's even though the flow is fitted (§3.7); latency resolving only at tick granularity, so the two-tick detectability floor is a property of that grid; and IOC-in-batch semantics inside the auction arm that are a modelling choice rather than an institutional given, which is part of why the batch-arm rent reduction stays suggestive rather than established.

Third, the ratio measured on the tape and the ratio swept in the simulation are not the same object. The tape's 0.0072 is a snipe against *aggregate* top-of-book depth — every maker resting at that level — whereas the simulation measures it against *one* maker's quote. The per-maker ratio is therefore bracketed below by the tape ratio and above by the tape ratio times the number of makers at the touch, a count no public feed reports, since depth arrives pre-aggregated with no maker identifiers. This matters because the inversion is a statement about the per-maker ratio: if the true value sits above the boundary at the relevant HFT count, the maker bears the tax after all. §4.8 resolves the bracket in the direction it warned of — splitting aggregate depth across M makers multiplies each maker's own ratio by M, and the maker-gains region at the tape band is gone by $M = 2$ — so what remains unobservable is M itself, and the question is now how many makers rest at the touch rather than which quantity the simulation sweeps. One methodological point belongs here rather than in a footnote: we apply no deflated-Sharpe-style multiple-testing correction at the cell level, because the surface reports every cell of a grid fixed before the runs rather than the survivors of a search, and a complete map selects nothing.

What survives all three is the structural point: the identity of the class bearing the latency tax

is not settled by the existence of the tax. It is a separate empirical question with a structural answer, and at realistic snipe sizes the answer is not the one the theory predicts.

5.3 Future work

The limitations above map directly onto an agenda for hardening these results.

Two items that formerly headed this agenda are discharged: §4.8 supplies competing makers with heterogeneous latencies, and §4.8 makes uninformed demand spread-elastic. Both were expected to threaten the incidence result and neither did in the way anticipated — competition removed the class-level inversion while leaving the within-class finding standing, and elasticity cost the maker-gains region one of eighteen tape-band cells. What stays open on that axis is elasticity above unit and its interaction with the maker-count surface, together with re-running both surfaces at $n = 100$, since each is mapped at $n = 20$ and resolves its boundary only to the width of the grid.

The first open priority is Experiment 5's real-data leg — the only piece of the §5.3 redesign agenda still open. The simulation side is complete (§4.5): time-based windows, quote-update and trade processes, and a passing placebo, yielding a validated null at the operating point. Running the same pipeline on the BTC tape requires one new component: a depth-depletion definition of a "gap" suited to a book that never empties (e.g., top-k depth falling below a percentile floor), after which the windowing, labelling, and placebo machinery apply unchanged. That leg is where the endogeneity hypothesis gets its empirical test, in the Filimonov–Sornette spirit, on flow where self-excitation is known to exist.

Second, the single-venue, single-asset scope should be relaxed. A multi-venue extension would let us study latency arbitrage across correlated instruments and order routing, the setting in which the mechanical-arbitrage evidence motivating BCS is most salient, and would test whether the flat-total-rent-with-rising-fragility pattern of Experiment 3 survives when snipers can compete across books rather than within one.

Third, the strategy spaces of both the market maker and the HFTs are deliberately narrow and should be enriched — for example, inventory-aware or quote-fade defences for the maker and more elaborate timing or order-type strategies for the snipers — to test how far the welfare decomposition and the batch-auction recovery are robust to more sophisticated play on both sides.

Fourth, the `adverse_sensitivity` check of §5.2 should be finished on its own terms, since it currently resolves the boundary's direction more confidently than its location. Two refinements would do it: re-running it at §4.7's $n = 100$ with the full ratio axis restored, the present $n = 20$ check pinning the boundary only to the width of the k-grid; and extending it to `spread_decay`, which enters the same stationary-spread relation and was held at its pre-registered value throughout. The same two apply to §4.8's surface, which is likewise an $n = 20$ map — its qualitative finding is carried by intervals that exclude zero, but the maker count at which the tape-band region closes is resolved only to the spacing of the M grid.

6. Conclusion

The finding we regard as this paper's most durable is about incidence *within* the liquidity-providing class, not about the class as a whole. At the snipe-to-depth ratio measured on the BTCUSDT-perpetual tape, the fastest maker in a competing set gains in every cell we tested at two and three makers, with its bootstrap interval excluding zero in all eighteen, while the slowest pays in all eighteen and funds both its faster competitor and the snipers (§4.8). The class-level answer is by contrast structure-dependent: a lone maker at the measured ratio profits from the arms race and noise traders bear the transfer, while adding a single competitor reverses that

and restores the incidence Budish, Cramton, and Shim predict (§4.8). Aggregate maker PnL is therefore a weighted sum over unlike participants rather than a verdict on liquidity provision, and the question "do liquidity providers bear the latency tax?" has no structure-free answer. The defensible statement is narrower and, we think, more useful: *slow* liquidity providers bear it. One condition on it deserves emphasis rather than a footnote: every maker here follows an imposed adverse-selection widening rule with an exogenous sensitivity parameter, so "the maker gains" is a property of that rule under sniping and not a claim about profit-maximizing liquidity providers who choose their spread endogenously — a maker free to re-optimize its quoting policy might capture more, or decline to quote at all, and nothing in this design bounds which.

That claim rests on a substrate built for it. We drive latency-differentiated snipers, an adverse-selection market maker, and noise traders through a C++ limit order book whose continuous-matching core is model-checked in TLA+, entirely through a pybind11 bridge, which turns the closed-market welfare identity into an audited, machine-precision check rather than a modelling assumption: the zero-sum residual closes to floating-point zero across all sixty-three cells of the single-maker surface, all 441 of the maker-count extension, and all 189 of the elasticity extension. Within that conserved frame the supporting results are that HFT rent is a near-exact zero-sum transfer rather than value destruction; that its cost scales smoothly in maker staleness with no detectable phase transition, though at our sample size the model comparison is close to indifferent between linear and segmented specifications; that as competing HFTs multiply, per-agent rent dissipates while total extracted rent stays flat, so the marginal cost of added competition surfaces as rising fragility rather than rising extraction; and that frequent batch auctions restore maker welfare monotonically above a minimum cadence while the per-tick auction backfires, with the parallel rent reduction suggestive rather than established.

The incidence result itself spans three structural axes. Sweeping snipe-to-quote ratio against HFT count under single-maker supply yields a monotone boundary rather than a single answer to "who pays": at large snipe sizes the maker bears the transfer and §4.1 replicates, while at small sizes it gains and noise traders bear the loss — through loss reduction rather than revenue capture, since the maker's noise-flow revenue falls when it widens and the benefit comes from the accompanying drop in adverse-fill losses, with inventory carry supplying a third to a half of the net (§4.7). Adding the maker-count axis removes that inversion as soon as a second maker enters, restoring the BCS prediction at the class level (§4.8); relaxing the inelastic-demand assumption up to unit elasticity leaves the picture intact, costing the eighteen tape-band cells only one, which turns indeterminate rather than reversing (§4.8). Measured on the perpetual's own tape the ratio sits at 0.0072 at the median, and the calibrated rent envelope of roughly 0.25–0.95 bp contains the ≈0.4 bp that Aquilina, Budish, and O'Neill measure from regulator-held exchange messages — envelope consistency between a simulated and a directly measured tax, not agreement on a number. The existence and sign of the rent were never in question here. What we add is that its incidence is a separate empirical question; that the answer depends on snipe size, sniper count, and how many makers supply the book; and that at the ratio the tape measures, the participants who bear the latency tax are the slow liquidity providers rather than liquidity providers as a class.

Code and Reproducibility

All experiment code lives under `bcs_research/` (agents in `bcs_research/agents/`, the latency scheduler and batch-auction scheduler in `bcs_research/simulation/`, metrics in `bcs_research/metrics/`, and the experiment drivers in `bcs_research/experiments/`), with the pybind11 bridge in `bcs_research/bindings/engine_bindings.cpp`. The matching engine is verified by the TLA+ specifications in `spec/` (notably `MatchingEngine.tla` and `Auction.tla`; see the TLC verification log in `docs/Verification.md` for the engine state-exploration record). Results are written as JSON under `bcs_research/results/experiments/`

and are reproducible by re-running the experiment scripts (`exp1_hardening.py` for the Experiment 1 results, `exp1_primary.py`, the Exp 2–4 sweep drivers, and `hawkes_bridge.py` for the §4.5 supplementary Hawkes pilot) at the seeds and operating point documented in §3.6.

References

Aquilina, M., Budish, E., & O'Neill, P. (2022). Quantifying the High-Frequency Trading "Arms Race". *Quarterly Journal of Economics*, 137(1), 493–564.

Baldauf, M., & Mollner, J. (2020). High-Frequency Trading and Market Performance. *Journal of Finance*, 75(3), 1495–1526.

Budish, E., Cramton, P., & Shim, J. (2015). The High-Frequency Trading Arms Race: Frequent Batch Auctions as a Market Design Response. *Quarterly Journal of Economics*, 130(4), 1547–1621.

Carlin, B. I., Lobo, M. S., & Viswanathan, S. (2007). Episodic Liquidity Crises: Cooperative and Predatory Trading. *Journal of Finance*, 62(5), 2235–2274.

Copeland, T. E., & Galai, D. (1983). Information Effects on the Bid-Ask Spread. *Journal of Finance*, 38(5), 1457–1469.

Du, S., & Zhu, H. (2017). What Is the Optimal Trading Frequency in Financial Markets? *Review of Economic Studies*, 84(4), 1606–1651.

Farmer, J. D., & Foley, D. (2009). The Economy Needs Agent-Based Modelling. *Nature*, 460(7256), 685–686.

Filimonov, V., & Sornette, D. (2012). Quantifying Reflexivity in Financial Markets: Toward a Prediction of Flash Crashes. *Physical Review E*, 85(5), 056108.

Foucault, T., Kozhan, R., & Tham, W. W. (2017). Toxic Arbitrage. *Review of Financial Studies*, 30(4), 1053–1094.

Glosten, L. R., & Milgrom, P. R. (1985). Bid, Ask and Transaction Prices in a Specialist Market with Heterogeneously Informed Traders. *Journal of Financial Economics*, 14(1), 71–100.

Hoffmann, P. (2014). A Dynamic Limit Order Market with Fast and Slow Traders. *Journal of Financial Economics*, 113(1), 156–169.

Kirilenko, A., Kyle, A. S., Samadi, M., & Tuzun, T. (2017). The Flash Crash: High-Frequency Trading in an Electronic Market. *Journal of Finance*, 72(3), 967–998.

Kyle, A. S. (1985). Continuous Auctions and Insider Trading. *Econometrica*, 53(6), 1315–1335.

LeBaron, B. (2006). Agent-Based Computational Finance. In *Handbook of Computational Economics*, Vol. 2 (pp. 1187–1233).

Menkveld, A. J. (2016). The Economics of High-Frequency Trading: Taking Stock. *Annual Review of Financial Economics*, 8, 1–24.

Menkveld, A. J., & Zoican, M. A. (2017). Need for Speed? Exchange Latency and Liquidity. *Review of Financial Studies*, 30(4), 1188–1228.

Wah, E., & Wellman, M. P. (2013). Latency Arbitrage, Market Fragmentation, and Efficiency: A Tactical Market Model. In *Proceedings of the 14th ACM Conference on Electronic Commerce (EC)*, 855–872.